



*Asociación Profesional de Cuerpos Superiores
de Sistemas y Tecnologías de la Información
de las Administraciones Públicas.*

Temas Específicos para la preparación de la Oposición al Cuerpo
Superior de Sistemas y Tecnologías de la Información de la
Administración del Estado.

TEMAS ESPECÍFICOS II: Tecnología básica

Tema 65. Arquitectura de desarrollo en la web. Desarrollo web front-end. Scripts de cliente. Frameworks. UX. Desarrollo web en servidor, conexión a bases de datos e interconexión con sistemas y servicios.

AUTOR: Javier Rincón Abel

Actualización 2017

ÍNDICE

1	INTRODUCCIÓN	4
2	ARQUITECTURA DE DESARROLLO EN LA WEB	5
2.1	ARQUITECTURAS WEB EN CAPAS	6
2.2	ARQUITECTURA WEB P2P	8
2.2.1	CARACTERÍSTICAS DE LAS APLICACIONES P2P	9
2.2.2	DIFICULTADES PARA LA COMUNICACIÓN P2P	9
3	TECNOLOGÍAS FRONT-END	10
3.1	HTML	10
3.2	XML	10
3.3	HOJAS DE ESTILO CSS	10
3.4	ADOBE ANIMATE / ADOBE FLASH PLAYER	11
3.5	JAVASCRIPT	11
3.6	ECMAScript	12
3.7	AJAX	12
3.8	JSCRIPT	12
3.9	VBScript	12
3.10	APPLETS	13
3.11	CONTROLES ACTIVE X	13
3.12	FRAMEWORKS	13
3.13	TIPOS DE APLICACIONES WEB	15
4	UX. EXPERIENCIA DE USUARIO	17
4.1	ESPECIFICACIONES FUNCIONALES	17
4.2	DISEÑO DE INTERACCIÓN	17
4.3	ARQUITECTURA DE LA INFORMACIÓN	18
4.4	DISEÑO DE INFORMACIÓN	18
4.5	DISEÑO DE NAVEGACIÓN	18
4.6	DISEÑO VISUAL APLICADO A LA WEB	19
4.7	USABILIDAD	19
4.8	USABILIDAD WEB	20
5	DESARROLLO WEB EN SERVIDOR	22
5.1	LENGUAJES Y TECNOLOGÍAS	22

5.1.1	PHP 7.1.9	22
5.1.2	JAVA EE 8.....	22
5.1.3	ASP 3.0	23
5.1.4	ASP.NET 5.0	23
5.1.5	RUBY 2.4.1.....	23
5.1.6	PYTHON 3.6.2.....	24
5.2	FRAMEWORKS BACK-END.....	24
5.2.1	FRAMEWORKS JAVA EE	24
5.2.2	FRAMEWORKS DE RUBY	25
5.2.3	FRAMEWORKS DE JAVASCRIPT	26
5.2.4	FRAMEWORKS DE PHP.....	27
5.2.5	FRAMEWORKS DE PYTHON	28
5.3	MOTORES DE PLANTILLAS	28
5.3.1	MOTORES DE PLANTILLAS JAVA EE.....	28
5.3.2	MOTORES DE PLANTILLAS RUBY.....	29
5.3.3	MOTORES DE PLANTILLAS JAVASCRIPT.....	29
5.3.4	MOTORES DE PLANTILLAS PHP	30
5.3.5	MOTORES DE PLANTILLAS PYTHON.....	30
6	OTROS ELEMENTOS EN LA ARQUITECTURA DE DESARROLLO WEB.....	31
6.1	SERVIDORES WEB	31
6.2	CONEXIÓN CON BASES DE DATOS	32
6.2.1	BASES DE DATOS SQL	32
6.2.2	FRAMEWORKS DE BASES DE DATOS	33
6.3	INTERCONEXIÓN CON SISTEMAS Y SERVICIOS.....	34
6.3.1	SERVICIOS WEB	35
6.3.2	FRAMEWORKS DE WEB SERVICES.....	35

1 Introducción

Este tema se plantea como objetivo cubrir un ámbito muy cambiante y muy amplio. Es muy cambiante ya que continuamente surgen y desaparecen tecnologías, mientras que es muy amplio ya que incluye ambos extremos de la arquitectura web. En las siguientes páginas se presentan contenidos que quizá tengan algún solape con los sucesivos temas, ya que estos últimos se ocupan más profundamente de cada una de las tecnologías front-end (cliente) o back-end (servidor) que aquí solo se mencionan. Por ello, y para evitar el posible solape, este tema se ha enfocado a modo de recorrido por las distintas tecnologías y frameworks más populares, pero sin entrar en profundidad en ninguna de estas tecnologías, intentando así no hacer excesivamente largo el tema y evitar el solapamiento de materias.

El tema está estructurado en varios bloques. El primero de ellos analiza las arquitecturas desde un punto de vista teórico para introducir los conceptos básicos y las características que hay que conocer. Ofrece una visión actual y sencilla de las arquitecturas web preparando el camino para los siguientes apartados. En el segundo bloque se analizan las tecnologías del lado del cliente, tanto los lenguajes como los frameworks más populares y que mayor productividad están demostrando. Le sigue un bloque que analiza conceptos esenciales para el desarrollo web en la actualidad como son la usabilidad y la experiencia de usuario. El siguiente bloque, más extenso, cubre las tecnologías con mayor difusión del lado servidor. Siguiendo el mismo esquema que para el lado cliente, en este bloque se analizan lenguajes y frameworks. El último bloque trata de algunos aspectos generales como son los servidores web o la conexión con las bases de datos y la conexión con servicios web, junto con los frameworks de cada uno de ellos.

Se ha intentado que el tema resulte útil para el primer ejercicio ya que permite conocer las tecnologías de forma breve y concisa sin entrar en muchos detalles, pero también para el cuarto ejercicio en el que se puede ver como un catálogo de tecnologías para componer soluciones a problemas que puedan surgir en los casos prácticos. Para desarrollar soluciones basadas en una tecnología concreta, se recomienda profundizar en ella, completando la información con el resto de temas que tratan más en profundidad cada una de estas tecnologías.

2 Arquitectura de desarrollo en la web

Las aplicaciones web son aquellas que permiten a los usuarios acceder a un servicio web alojado en un servidor a través de una red (Internet o intranet) mediante el uso de un navegador. Estas aplicaciones web son cada vez más populares debido a las ventajas que ofrece un navegador web, al convertirse en un cliente ligero independiente del sistema operativo. Otra ventaja importante es que este tipo de aplicaciones facilita las tareas a realizar para la actualización y el mantenimiento, ya que la distribución de nuevas versiones de una aplicación no implica el tener que distribuir el nuevo software ni instalar y configurar cada equipo cliente. Algunos ejemplos de aplicaciones que siguen esta aproximación son: el correo web, los blogs, tiendas online, bancos online y periódicos digitales. Estos son algunos de los más conocidos pero cada día aparecen nuevos servicios.

Otra característica que tienen las aplicaciones desarrolladas en arquitecturas web es la capacidad de contener elementos de comunicación dinámicos para los usuarios. Es decir, que las acciones de la interacción del usuario con las aplicaciones transforman la información obtenida por el propio usuario trayendo nueva información del servidor, pero también pueden alterar las vistas de otros usuarios en función de las acciones y su implicación. Acciones como rellenar un formulario, consultar una base de datos o descargar un documento generara distinta respuesta por parte del servidor.

En los comienzos, cuando empezaron a desarrollarse las aplicaciones de tipo cliente-servidor, la aproximación que se usó fue la de instalar de forma independiente en cada ordenador cliente un programa que servía como interfaz de usuario y que se comunicaba con el servidor intercambiando datos mediante diferentes protocolos y tecnologías. Este enfoque fue evolucionando hacia otro en el que el cliente ya no se instala, si no que las aplicaciones web consisten en cargar páginas con formatos interpretables por un navegador. Esta aproximación moderna se sustenta en estándares tanto para las interfaces (HTML, CSS, Javascript) como para las comunicaciones (TCP/IP, DNS, HTTP). Durante la sesión de navegación de un usuario el navegador realiza las solicitudes al servidor y muestra la interpretación de los datos de las páginas devueltas, actuando dicho navegador como cliente único para cualquier aplicación web.

La evolución de las tecnologías estándar procesables por los navegadores ha permitido que la experiencia del usuario sea muy similar a la experiencia que se tendría al instalar un cliente específico, aunque las interfaces web presentan algunas limitaciones en las funcionalidades que se dan al usuario. Si bien es cierto que cada vez son menos, aún existen una serie de funcionalidades comunes en las aplicaciones de escritorio como dibujar en la pantalla o arrastrar-y-soltar que no están soportadas por las tecnologías web estándar. Los desarrolladores web generalmente utilizan lenguajes interpretados (scripts) en el lado del cliente para añadir más funcionalidades, especialmente para ofrecer una experiencia interactiva que no requiera recargar la página cada vez (lo que suele resultar molesto a los usuarios). Por este motivo, se han desarrollado tecnologías para coordinar estos lenguajes con las tecnologías en el lado del servidor como AJAX que permitan complementar la funcionalidad estándar e igualar la experiencia a la de los clientes de aplicaciones instalables y que se analizarán más adelante en este tema.

Como hemos mencionado en la arquitectura web existen dos actores principales con características que les identifican y que se podrían resumir en las siguientes.

Cliente:

- Inicia solicitudes o peticiones. Tiene un papel activo en la comunicación.
- Espera y recibe las respuestas del servidor.
- Puede conectarse a varios servidores a la vez.
- Interactúa directamente con los usuarios finales mediante una interfaz gráfica de usuario.

Servidor:

- Al iniciarse espera a que le lleguen las solicitudes de los clientes. Desempeña un papel pasivo en la comunicación.
- Tras la recepción de una solicitud, la procesa y luego envía la respuesta al cliente.

- Aceptan conexiones desde un gran número de clientes (en ciertos casos el número máximo de peticiones puede estar limitado).
- No es frecuente que interactúen directamente con los usuarios finales.

Después de introducir el concepto de aplicación web vamos a enumerar de forma más sistemática las características de este tipo de arquitectura:

- No precisa instalar un software especial en los clientes para acceder a los nuevos servicios solo es necesario un navegador, que viene con el propio sistema operativo, como por ejemplo: Internet Explorer, Firefox, Opera, Chrome, etc.
- Actualizar la versión de software en los clientes no es un gran coste. Al ser el servidor quien ejecuta la mayor parte del código de la aplicación y suministra de forma centralizada las vistas a los navegadores, no hay que instalar nada en los puestos de trabajo, ya que la actualización se realiza en el servidor y automáticamente la ven todos los usuarios.
- Facilita que todos los clientes dispongan de la última versión de forma inmediata. Esto permite evitar que algunos clientes mantengan versiones desactualizadas o incompatibles que podrían originar problemas de pérdida de información o inconsistencias en los datos.
- La información está disponible de forma única y centralizada. El disponer de la información en un servidor centralizado permite que esté siempre disponible y mejora el acceso concurrente.
- Como consecuencia de mantener la información centralizada, se simplifica la gestión de aspectos como la gestión de las copias de seguridad y el control de acceso a los datos. Si los datos se encontrasen dispersos en cada uno de los clientes sería muy difícil garantizar que no existen pérdidas o que los backups son completos y consistentes. Además, en caso de un incidente la recuperación de los sistemas a través de las copias de seguridad también es más sencillo.
- Como consecuencia de tener que acceder a un servicio remoto a través de las redes el servicio se puede ver afectado por la congestión de las redes provocando degradaciones en caso de mucho tráfico.
- Por otra parte, el contar con un único repositorio de datos provoca que en caso de fallo del servidor se produce una parada completa del servicio, si bien es cierto que esta circunstancia se puede paliar redundando los servicios y balanceando los servicios en caso de fallo, pero deben ser tenidos en cuenta y planificados.
- Ofrece la posibilidad de acceder a la aplicación web desde cualquier parte del mundo. Al tener el servicio alojado en un servidor remoto lo único que se necesita es un equipo con un navegador y con acceso a la red del servidor. De este modo la movilidad del cliente ofrece una mayor flexibilidad.
- Al tratarse de clientes ligeros los navegadores no requieren de grandes recursos en el lado del cliente, esto permite que no sea necesario reemplazar los equipos clientes con tanta frecuencia. Es decir, dado que la lógica y los procesos más pesados corren en el servidor la carga de trabajo del cliente es muy pequeña y la obsolescencia se ve reducida.

2.1 Arquitecturas web en capas

Dado que la arquitectura web se estructura en capas vamos a realizar el análisis teórico de cómo se pueden distribuir las funciones entre cada uno de los elementos constituyendo las capas. A lo largo de este epígrafe se analizará esta distribución entre las distintas capas y sus características.

El término arquitectura cliente-servidor es utilizado comúnmente para hablar de la arquitectura de las aplicaciones web, y también los términos arquitectura en 2 y 3 capas se aplican al mismo concepto, por lo que resulta importante hacer una distinción. Así pues, se entiende por arquitectura física a la topología de la aplicación, mientras que de forma independiente una aplicación está dividida en componentes desde el punto de vista lógico que llamamos capas.

Las capas como elementos lógicos de las aplicaciones son unidades altamente cohesivas, con responsabilidades de alto nivel, bien definidas y autocontenidas. Cuando establecemos una organización del software de una aplicación en términos de componente de capas, estamos hablando

de la arquitectura lógica de la aplicación. Arquitectura cliente-servidor habla de la topología del software, mientras que decir que su arquitectura es en capas habla de su arquitectura lógica.

Las aplicaciones web presentan tres aspectos fundamentales, ya que por un lado debe hacer que los **datos** sean persistentes (**D**), al mismo tiempo debe procesarlos de forma acorde a la **lógica de negocios** (**L**), y finalmente debe **presentarlos** adecuadamente a los usuarios (**P**).

En las aplicaciones en una única capa (P+L+D) no existe distinción ni separación lógica entre estos tres aspectos. Este tipo de aplicaciones son muy grandes, difíciles de mantener, distribuir, incompatibles con la arquitectura web, pesadas y con gran consumo de recursos, por lo que no se van a abordar si no que se dejaran fuera de este análisis.

En primer lugar se van a analizar las arquitecturas web en dos capas que suponen la división de las funciones o responsabilidades entre dos unidades lógicas. En segundo lugar, se analiza la distribución en tres capas que implica que cada unidad lógica se encarga de una de las responsabilidades.

1. ARQUITECTURA WEB EN DOS CAPAS

Una arquitectura web en dos capas distribuye la aplicación en dos componentes lógicos como hemos mencionado antes, y según sea la distribución de responsabilidades de cada componente dará lugar a cada una de las variantes de esta arquitectura.

Esta arquitectura web en dos capas fue la primera que surgió como consecuencia de la aparición de la arquitectura cliente-servidor. Esta topología permite distribuir la carga de la aplicación a dos sistemas diferentes, lo que llevó de forma natural a distribuir las responsabilidades de la misma en dos unidades lógicas.

Arquitectura web tipo P+L/D

La primera variante de la arquitectura consiste en retirar el manejo de datos de la aplicación. Esto permite a varios clientes utilizar el mismo conjunto de datos. La presentación y la lógica de negocio (P+L) es una unidad lógica por sí, donde el manejo de interfaz de usuario y el manejo de la lógica no se los distingue como módulos independientes. Típicamente P+L se encuentra en el cliente, mientras que D se encuentra en el servidor. Se consideran como interfaces pesados al tener mucha carga de trabajo ya que contienen la lógica, pero suelen ser más rápidos en las comunicaciones.

Arquitectura web tipo P/L+D

Esta arquitectura tiene en cuenta que como en las aplicaciones web la lógica del cliente es la misma se puede recurrir a la solución de llevarla al servidor para unificarla. De esta forma, la lógica de la aplicación se encuentra embebida al manejo de la persistencia de datos. En este tipo de aplicaciones la lógica resuelve los problemas de persistencia encargándose ella misma de dicha tarea, no necesariamente utilizando un gestor de base de datos, o embebiendo toda la lógica de negocios en el mismo.

Arquitectura web tipo P+L/L+D

La última variante consiste en el equilibrio de repartir la tarea de la lógica, una parte junto a la interfaz de usuario, y otro junto al manejo de persistencia de datos. Estas aplicaciones son similares a las que tienen arquitectura P+L/D, que tienen implementada parte de la lógica en procedimientos almacenados en el manejador de la base de datos.

Algunas de las desventajas más importantes de la arquitectura web en dos capas son:

- La lógica de la aplicación no puede ser reutilizada ya que está ligada o a la interfaz de usuario o al manejo de persistencia de datos.
- El entorno cliente tiene recursos más limitados por lo que los desarrolladores deben estar entrenados para optimizar la aplicación de forma que pueda ser utilizada en dichos entornos.
- Existe una sobrecarga en el uso de la red: dado que el procesamiento de los datos se realiza en el cliente, gran cantidad de información debe ser transmitida desde el servidor.

2. ARQUITECTURA WEB EN TRES CAPAS

La variante de la arquitectura web en dos capas de tipo P/L+D, dio lugar a la arquitectura web en tres capas. Como hemos visto, el hecho de que la lógica de negocio y el manejo de persistencia sean una

unidad presentaba desventajas importantes en el caso de que fuese necesario cambiar la parte del gestor de base de datos al estar fuertemente enlazada a la lógica.

En esta arquitectura web en tres capas la lógica de la aplicación ocupa una capa intermedia; por tanto, está separada de los datos y de la interfaz de usuario (P/L/D). Los procesos pueden ser administrados y desplegados en forma independiente, sin relación con la interfaz de usuario o con el gestor de base de datos. Las aplicaciones web en tres capas son de más fácil ampliación y más robustas y flexibles, con la ventaja añadida de que pueden integrar datos de múltiples fuentes.

No hay que perder de vista que cuando hablamos de tres capas lógicas no implica que las tres capas no puedan estar alojadas en el mismo servidor, ya que lo importante es que la aplicación web esté claramente estructurada y que exista una buena definición de los límites y la interacción entre las distintas capas.

Una vez vistas las variantes de la arquitectura de tres capas, vamos a identificar las responsabilidades clásicas de cada una de las capas que se han mencionado.

1. **Capa de presentación:** Tiene como responsabilidad la presentación de los datos, recibiendo los eventos de los usuarios y controlando la interfaz de usuario.
2. **Capa de lógica de negocios:** Esta capa no está presente en la arquitectura en 2 capas en forma explícita. En esta capa se sitúan los objetos de negocio que implementan las reglas de negocio y están disponibles para la capa de presentación. Además, protege del acceso directo a los datos desde la capa de presentación
3. **Capa de persistencia:** Es responsable del almacenamiento de los datos, recuperación y almacenamiento de la información de la aplicación web.

Como ventajas de la arquitectura web en tres capas podemos destacar que cuenta con una separación clara del interfaz de usuario con la lógica de la aplicación. Esta separación permite disponer de diferentes presentaciones posibles que accedan a la misma lógica. Además, los cambios en el almacenamiento de la información no tienen impacto sobre la capa de presentación.

2.2 Arquitectura web P2P

Las arquitecturas tipo cliente-servidor no son las únicas que podemos plantear para el desarrollo de aplicaciones web. Existe un planteamiento alternativo que sugiere una distribución de roles entre las máquinas de forma diferente, que son las llamadas redes P2P o una red peer-to-peer (P2P, por sus siglas en inglés)

Esta aproximación consiste en una red de ordenadores que funciona sin clientes ni servidores fijos, de tal forma que todos los nodos involucrados en la solución se comportan como iguales entre sí. Es decir, actúan simultáneamente como clientes y servidores respecto a los demás nodos de la aplicación. Las aplicaciones web sobre redes P2P permiten el intercambio directo de información entre los ordenadores interconectados.

Las aplicaciones web sobre redes peer-to-peer aprovechan, administran y optimizan el uso del ancho de banda de los demás usuarios de la red por medio de la conectividad entre los mismos, y obtienen así más rendimiento en las conexiones y transferencias que con algunos métodos centralizados convencionales, donde una cantidad relativamente pequeña de servidores provee el total del ancho de banda y recursos compartidos para un servicio o aplicación.

Este tipo de aplicaciones web resultan especialmente útiles para diversos propósitos como compartir o distribuir ficheros de cualquier tipo, pero también suele usarse en telefonía VoIP para hacer más eficiente la transmisión de datos en tiempo real.

Estas soluciones están especialmente indicadas para aquellas aplicaciones y servicios que requieran una enorme cantidad de recursos. El ejemplo más conocido es el intercambio y búsqueda de ficheros. Quizás sea la aplicación más extendida de este tipo de redes. Algunos ejemplos son BitTorrent o emule, pero también es usado por sistemas de ficheros distribuidos, como CFS o Freenet.

También se usa en sistemas para proporcionar anonimato o sistemas de telefonía por Internet, como Skype. Otros usos son los cálculos científicos que procesen enormes bases de datos, como los procedimientos bioinformáticos o las aplicaciones de transacciones entre particulares para monedas virtuales como Bitcoin.

2.2.1 Características de las aplicaciones P2P

A continuación vamos a analizar de forma breve las características más importantes de las aplicaciones sobre redes P2P.

1. Escalabilidad

Las aplicaciones P2P tienen un alcance mundial y cuantos más nodos estén conectados a una red P2P, mejor será su funcionamiento. Así, cuando los nodos llegan y comparten sus propios recursos, los recursos totales del sistema aumentan. Esto es diferente en una arquitectura del modelo cliente-servidor con un sistema fijo de servidores, en los cuales la adición de clientes podría significar una transferencia de datos más lenta para todos los usuarios.

2. Robustez

La naturaleza distribuida de las aplicaciones peer-to-peer también incrementa la robustez en caso de haber fallos mediante la réplica de los datos hacia múltiples destinos, permitiendo a los nodos encontrar la información sin hacer peticiones a ningún servidor centralizado de indexado. En el último caso, no hay ningún punto singular de fallo en el sistema lo que hace que resista mejor.

3. Descentralización

Estas redes por definición son descentralizadas y todos los nodos son iguales. No existen nodos con funciones especiales, y por tanto ningún nodo es imprescindible para el funcionamiento de la red.

4. Distribución de costes

Se comparten o donan recursos a cambio de recursos. Según la aplicación de la red, los recursos pueden ser archivos, ancho de banda, ciclos de proceso o almacenamiento de disco. Cada nodo cede recursos para poder unirse al sistema.

5. Seguridad

Esta característica requiere identificar y evitar los nodos maliciosos, evitar el contenido infectado, evitar el espionaje de las comunicaciones entre nodos, la creación de grupos seguros de nodos dentro de la red y la protección de los recursos de la red.

2.2.2 Dificultades para la comunicación P2P

Existe el problema de que la mayor parte de los nodos de Internet no disponen de una dirección IP fija por lo que para el correcto funcionamiento de una aplicación web P2P, hay que resolver dos problemas fundamentales: cómo se encuentra un nodo que ya esté conectado a la red P2P y cómo se conectan los nodos sin dirección IP pública entre ellos.

Para resolver el primer problema, la solución habitual es realizar una conexión a un servidor inicial con dirección bien conocida que el programa P2P tiene almacenada. Este servidor inicial se encarga de mantener una lista con las direcciones de otros nodos que están actualmente conectados a la red. Tras esto, los clientes ya tienen información suficiente para entrar en la red y pueden intercambiar información con otros nodos, ya sin intervención de los servidores iniciales.

En cuanto a la forma de resolver el problema de conexión cuando los nodos no tienen dirección pública, estos se conectan a través de otro nodo que funciona como proxy de la conexión. Los dos nodos se conectan al proxy y éste envía la información que llega de uno al otro. Cualquier nodo con una dirección IP pública puede ser escogido como proxy de una conexión entre dos nodos. Por ejemplo, en la red Skype a través de nuestro ordenador pueden pasar conversaciones de otras personas. En estos casos, es imprescindible la implementación de algún mecanismo de seguridad para evitar que los proxies puedan llegar a entender la comunicación entre los dos nodos.

3 Tecnologías Front-end

Las tecnologías que se sitúan en este extremo de la arquitectura están orientadas a formatear la información que se ha recibido de una petición en concreto al servidor, pero también se encargan de mantener la información entre las distintas llamadas constituyendo las sesiones, y de realizar las nuevas llamadas para traer nueva información que ofrecer al cliente.

Algunas de las tecnologías que aquí se mencionan son tratadas en profundidad en el tema específico dedicado a HTML, CSS y XML, pero se incluyen aquí por su importancia y por constituir la base de las tecnologías front-end.

3.1 HTML

HTML (Hypertext Markup Language) o lenguaje de marcado hipertextual, es el código creado para el desarrollo de páginas web. En líneas generales, este código permite describir la estructura de un sitio en Internet, dar formato al contenido, insertar imágenes y generar vínculos con otras páginas.

El código HTML se compone de etiquetas que se indican entre corchetes angulares `< >`. Las etiquetas funcionan como categorías u órdenes que definen cómo se organizará y qué características tendrá la información de nuestra página web. Para dar formato al contenido y crear hipervínculos en el cuerpo de la página (enmarcado entre las etiquetas `<body>` y `</body>`) también se utilizan etiquetas que indican a los navegadores web cómo debe visualizarse la página.

La versión actual es HTML 5 y el organismo encargado de su desarrollo es el World Wide Web Consortium (W3C).

3.2 XML

XML (eXtensible Markup Language) o lenguaje de marcado extensible, es un metalenguaje extensible de etiquetas desarrollado por el World Wide Web Consortium al igual que HTML. XML no es realmente un lenguaje en particular, sino una manera de definir lenguajes para diferentes necesidades, por eso se le llama meta-lenguaje. Algunos de estos lenguajes que usan XML para su definición son SVG, MathML, o como no el XHTML.

XML no ha nacido sólo para su aplicación en Internet, sino que se propone como un estándar para el intercambio de información estructurada entre diferentes plataformas. El XML se usa en bases de datos, editores de texto, hojas de cálculo y casi cualquier cosa imaginable.

El XML es una tecnología sencilla, pero tiene a su alrededor otras que la complementan o se basan en ella y la hacen mucho más grande y con unas posibilidades mucho mayores. Tiene un papel muy importante en la actualidad ya que permite la compatibilidad entre sistemas para compartir la información de una manera segura, fiable y fácil. Es una herramienta que permite el intercambio estructurado de información facilitando la interoperabilidad mediante la definición de formatos de datos, ya que es un elemento básico en muchas de las tecnologías que se verán a continuación.

3.3 Hojas de estilo CSS

CSS (Cascading Style Sheets) se utiliza para dar estilo a documentos HTML y XML, permitiendo la separación entre el contenido (HTML) y la presentación (CSS). Es decir, el código HTML permite dar formato al contenido de una página web mediante algunas etiquetas como por ejemplo negrita, subrayado, itálica o títulos, pero tiene también muchas limitaciones a la hora de crear páginas modernas. Impulsado por el desarrollo y la expansión de Internet comenzaron a demandarse tecnologías que permitieran ver el contenido de una página web en distintos dispositivos, desde un monitor de ordenador hasta un teléfono móvil o una televisión.

Los estilos definen la forma de mostrar los elementos HTML y cualquier cambio en el estilo marcado para un elemento en la CSS afectará a todas las páginas vinculadas a ella en las que aparezca ese

elemento. De esta forma, CSS permite controlar el estilo y formato de múltiples páginas web al mismo tiempo.

Las definiciones del lenguaje CSS funcionan a base de reglas, es decir, mediante declaraciones sobre el estilo de uno o más elementos. Una regla tiene dos partes: un selector y la declaración, estando esta última compuesta por una propiedad y el valor que se le asigne.

El selector funciona como enlace entre el documento y el estilo, especificando los elementos que se van a ver afectados por esa declaración. La declaración es la parte de la regla que establece cuál será el efecto.

La última versión es CSS3 y en ella hay muchas características nuevas que mejoran las posibilidades gráficas de las presentaciones en las páginas web: animaciones, efectos, etc. El organismo responsable del mantenimiento de este estándar es también el W3C.

3.4 Adobe Animate / Adobe Flash Player

Adobe Animate es una aplicación de creación y manipulación de gráficos vectoriales con posibilidades de manejo de código mediante un lenguaje de scripting llamado ActionScript. Animate es un estudio de animación que trabaja sobre "fotogramas" y está destinado a la producción y entrega de contenido interactivo que puede ser usado también en páginas web.

Animate es el entorno de desarrollo y Flash Player es el reproductor utilizado para visualizar los archivos generados que puede ser usado como plugin del navegador. En otras palabras, Adobe Animate crea y edita las animaciones o archivos multimedia y Adobe Flash Player las reproduce.

Es una tecnología propietaria y cerrada, que actualmente es desarrollada y comercializada por Adobe Systems Incorporated. Aunque es gratuita para los usuarios de Flash Player, los desarrolladores y servidores que usen ciertas características tienen que pagar licencia.

Fue una tecnología muy usada principalmente para incrustar contenido multimedia interactivo en páginas web durante muchos años, ya que era la única forma de tener interactividad, animaciones, vídeos, juegos... en Internet, pero con la aparición de las nuevas capacidades de HTML5 de muchas librerías Javascript, se considera que Flash no es eficiente ni abierta y, por tanto, no es el futuro de Internet. Finalmente, Adobe lo acabó reconociendo y planea terminar de soportar esta plataforma para 2020.

3.5 Javascript

Javascript es un lenguaje de scripting basado en objetos no tipado y ligero, utilizado para acceder a objetos en aplicaciones. Principalmente, se utiliza integrado en un navegador web permitiendo el desarrollo de interfaces de usuario mejoradas y páginas web dinámicas.

Javascript ha tenido influencia de múltiples lenguajes y se diseñó con una sintaxis similar al lenguaje de programación Java, aunque es fácil de utilizar para personas que no programan. Todos los navegadores modernos interpretan el código Javascript integrado dentro de las páginas web. Para interactuar con una página web se provee al lenguaje Javascript de una representación estándar del código HTML de la página web.

Javascript es interpretado, por lo que no requiere compilación. Además, es un lenguaje orientado a eventos y cuando un usuario pincha sobre un enlace o mueve el puntero sobre una imagen se produce un evento, que mediante Javascript se pueden desarrollar scripts que ejecuten acciones en respuesta a estos eventos.

Hoy en día, el lenguaje está aumentando su popularidad, pasando de ser considerado una herramienta para aficionados a uno de los lenguajes más usados actualmente, con presencia tanto en los clientes, como en los servidores web como se verá más adelante en este tema.

3.6 ECMAScript

ECMAScript es una especificación de lenguaje de programación Javascript publicada por ECMA International, que comenzó su desarrollo en 1996 y que actualmente está aceptado como el estándar ISO 16262. ECMAScript define un lenguaje de tipos dinámicos ligeramente inspirado en Java y otros lenguajes del estilo de C. Además, soporta algunas características de la programación orientada a objetos mediante objetos basados en prototipos y pseudo-clases.

La mayoría de navegadores web incluyen una implementación del estándar ECMAScript. La versión del estándar ECMAScript 5 es la más usada actualmente, aunque en junio de 2015 se cerró y publicó la versión ECMAScript 6.

3.7 AJAX

AJAX (Asynchronous Javascript And XML) o Javascript asíncrono y XML, es una técnica de desarrollo web creada con el fin de implementar aplicaciones interactivas que se ejecutan en el navegador de los usuarios y mantienen comunicación asíncrona con el servidor en segundo plano. De esta forma es posible realizar cambios sobre la misma página sin necesidad de refrescarla, lo que significa aumentar la interactividad, velocidad y usabilidad en la misma.

Cuando se usa el modelo AJAX, las aplicaciones funcionan mucho más rápido, ya que las interfaces de usuario se pueden actualizar por partes sin tener que actualizar toda la página completa. Por ejemplo, al rellenar un formulario de una página web, con AJAX se puede actualizar la parte en la que se elige el país de residencia sin tener que actualizar todo el formulario o toda la página web completa.

3.8 JScript

JScript es la implementación de Microsoft de ECMAScript y la versión más reciente es JScript .NET, que está basado en la versión 4 del estándar ECMAScript, aunque en la práctica JScript presenta incompatibilidades con dicha especificación y con Javascript, haciendo las sentencias escritas en JScript ilegibles para otros navegadores distintos a Internet Explorer. Por este motivo, los scripts que deban ser ejecutados en el cliente siempre deberían ser programados en Javascript para que funcione en todos los navegadores.

Las diferencias entre Javascript y JScript son muy importantes, por ejemplo, mientras que Javascript no es sensible a las mayúsculas y minúsculas para los nombres de función estándar, JScript sí lo es. Otro problema es el de la diferencia entre la jerarquía de objetos, que dificulta el acceso a los objetos en la página web.

JScript es un lenguaje en el que no se necesita declarar los tipos de datos. Esto significa que no es necesario declarar explícitamente los tipos de datos de las variables. Además, en muchos casos JScript realiza conversiones de forma automática cuando es necesario.

3.9 VBScript

Microsoft Visual Basic Scripting, pertenece a la familia de lenguajes de Visual Basic la apariencia y forma de programación en VBScript es muy similar al lenguaje de programación de Microsoft Visual Basic o Visual Basic para aplicaciones.

VBScript se comunica con las aplicaciones principales mediante ActiveX Scripting. La ventaja principal de VBScript es la integración total al sistema operativo Microsoft, gracias a la cual crea aplicaciones web muy sofisticadas. La desventaja principal de VBScript también es su integración total al sistema operativo de Microsoft, dado que no corre sobre otros sistemas operativos ni sobre otros navegadores. Es exclusivo para usuarios de Internet Explorer y Windows.

Actualmente Microsoft no ha puesto a disposición pública nuevas versiones del lenguaje desde la última 5.8, en favor de la tecnología .NET en la que se incluye el lenguaje hermano Visual Basic, dentro del entorno de ejecución de la plataforma.NET (CLR, o Common Language Runtime).

3.10 Applets

Un applet es un programa de Java incrustado en una página HTML. Es importante entender la diferencia entre un applet y una aplicación. Los applets se diseñan específicamente como objetos incrustados en páginas web y no se ejecutan fuera de este contexto. Se descargan como documentos HTML y se ejecutan en el cliente.

Los applets que se cargan desde la red no pueden leer ni escribir archivos del sistema cliente. Además, incorporan otras medidas de seguridad adicionales para evitar que se utilice la máquina cliente como pasarela, mediante la habilitación de conexiones de Internet únicamente a través del host de origen. Tampoco se permite que los applets puedan iniciar aplicaciones instaladas en el ordenador cliente ni acceder a ellas. Se prohíbe también que carguen bibliotecas o definan llamadas de métodos nativos, lo cual impide el acceso al sistema operativo y a sus recursos.

Java restringe el acceso a los recursos locales cuando se presentan applets que no son de fiar. Todos los programas se ejecutan de forma predeterminada dentro de la “caja de arena” (sandbox) de Java. Estas restricciones de seguridad también limitan las posibilidades de uso de los applets de Java. Está establecido que no se pueden imprimir ni guardar documentos. Para permitir estas opciones, hay que pedir permiso a los clientes. Así mismo, para poder incorporar la capacidad de acceso a los recursos locales es necesario firmar el applet. Mediante este procedimiento se revela la identidad del programador. Este procedimiento de firma y confirmaciones por parte de los clientes hace que los applets de Java sean muy seguros, aunque es válido únicamente para los que se descargan de la red, mientras que aquellos que están ubicados en el disco local tienen estas opciones habilitadas automáticamente.

3.11 Controles ActiveX

Los controles ActiveX son componentes (u objetos) que se pueden insertar en una página web o una aplicación para proporcionar características sofisticadas y animaciones. Los controles ActiveX son pequeños y versátiles y ofrecen una puerta abierta hacia la creación de contenido web espectacular. Pueden ser escritos en casi todos los lenguajes de programación, incluyendo Java, C++ y Microsoft Visual Basic. Los controles ActiveX se pueden reutilizar con facilidad en otras páginas web o aplicaciones.

Si comparamos los applets de Java con ActiveX, veremos que ambas tecnologías ofrecen la posibilidad de agregar funcionalidades a páginas web estáticas sin la necesidad de conexiones de servidor adicionales. ActiveX y Java refuerzan el control de la seguridad para los datos del ordenador del cliente haciendo necesario que los usuarios tengan que aceptar permitir el acceso para garantizar la seguridad y la privacidad de los datos.

Por tanto, la principal diferencia entre un control ActiveX y un applet de Java reside en el nivel de daños que se pueden provocar al aceptar programas desconocidos. El usuario no permite el acceso a los recursos locales por el sólo hecho de aceptar un applet de Java, pero sí lo permite con los controles ActiveX desde el momento en que acepta la descarga. La diferencia consiste en que, en el caso de Java, el usuario no autoriza el acceso, mientras que en ActiveX sí. Para evitar riesgos, se recomienda rechazar los programas que provienen de servidores de Internet desconocidos.

3.12 Frameworks

Podemos entender un framework como una estructura de soporte definida sobre la que otras aplicaciones software pueden ser organizadas y desarrolladas. Esta definición puede incluir programas soporte, librerías y scripts de código y cualquier tipo de servicio que permita unir los diferentes componentes de una aplicación.

Centrándonos en frameworks de aplicaciones web, se pueden definir como plataformas modulares y reutilizables que pueden ser especializadas para producir aplicaciones web personalizadas. Generalmente, estas plataformas implementan el paradigma Modelo-Vista-Controlador (MVC) e integran servicios como búsquedas y control de versiones.

Entre los motivos por los que es tan popular el uso de frameworks, destacan el hecho de que se basan en arquitecturas abiertas estandarizadas y también, porque suponen un gran ahorro de tiempo en inclusión de servicios básicos de cualquier aplicación web, como una capa de persistencia para el acceso a una base de datos, gestión de usuarios, gestión de grupos y autorizaciones de acceso, con lo que aumenta la productividad y la calidad en el desarrollo web.

Dado que el lenguaje más usado para la implementación dinámica del lado cliente es Javascript, los frameworks que se analizan a continuación están centrados en este lenguaje que es el más desarrollado, potente y ampliamente soportado en todos los navegadores.

1. jQuery 3.2.1

jQuery es una biblioteca o framework gratuito de Javascript, que permite la realización de programas Javascript de una forma simple y sencilla, creando páginas web de las aplicaciones dinámicas complejas. jQuery es una librería Javascript muy rápida y muy ligera que simplifica el desarrollo de la parte de cliente de las aplicaciones web.

jQuery tiene diversas prestaciones, entre las que destacan por un lado el permitir al desarrollador despreocuparse de la compatibilidad de los scripts con los distintos navegadores existentes. Por otro lado, permite una mayor facilidad en la creación de aplicaciones del lado cliente, es decir, interfaces de usuario, efectos dinámicos o aplicaciones que hacen uso de AJAX.

2. jQuery Mobile 1.4.5

jQuery Mobile es un framework Javascript basado en HTML5 para el diseño de la interfaz de usuario, diseñado para hacer aplicaciones web “responsive” accesibles desde cualquier Smartphone, Tablet o dispositivo de escritorio. Se encuentra en continuo desarrollo por el equipo de jQuery. Es compatible con otros frameworks y plataformas como PhoneGap o Worklight.

Basa su ventaja sobre los demás frameworks en su capacidad para adaptarse a cualquier tipo de dispositivo, así como plataformas móviles. Se construye sobre un núcleo jQuery por lo que es de fácil aprendizaje para los programadores familiarizados con la sintaxis jQuery. Además, cuenta con la herramienta “builder” para generar un archivo Javascript personalizado así como hojas de estilos también personalizadas para no usar más código del necesario.

3. Bootstrap 4

Bootstrap es una herramienta que proporciona de forma gratuita una librería de CSS que se puede añadir directamente en los proyectos y que permite que se haga uso de las diferentes clases que tiene definidas en el código, así como de un framework CSS especializado en diseños web para dispositivos móviles.

Aunque es un framework que resulta complejo para usuarios sin experiencia en la programación de CSS, ofrece un mundo enorme de posibilidades en cuanto al formato se refiere y posee un potente sistema de grids o columnas. Dicho sistema divide la pantalla en doce partes, y las posibilidades de tamaño de pantalla en cuatro rangos de resoluciones, y de esta forma, si a cada elemento web de la aplicación se le asignan cuantas de esas doce partes se quiere que ocupe en cada uno de los posibles rangos de resoluciones, se podrá modelar a la perfección la forma en que la aplicación web se visualiza, aunque tenga una considerable carga de trabajo.

4. AngularJS 1.6.6

Este framework permite escribir una aplicación de manera sencilla, que con solo leerla podríamos entender qué es lo que se quiere lograr sin esforzarnos demasiado. Angular permite construir aplicaciones web modernas e interactivas mediante el aumento del nivel de abstracción entre el desarrollador y las tareas de desarrollo de aplicaciones web más comunes. Extendiendo el tradicional HTML con etiquetas propias que llamamos directivas. Se podría decir que utiliza el patrón MVC (Model-View-Controller), aunque ellos mismos lo definen como un patrón distinto denominado MVW (Model-View-Whatever (whatever works for you)). La idea de la estructura que propone no es otra que presentar una organización en el código, donde el manejo de los datos (Modelo) estará separado de la lógica (Controlador) de la aplicación, y a su vez la información presentada al usuario (Vistas) se encontrará totalmente independiente.

Las plantillas de AngularJS nos permiten crear aplicaciones de una sola página, de tal forma que podemos cargar diferentes partes de la aplicación sin tener que recargar todo el contenido en el navegador. Al mismo tiempo un motor de plantillas genera contenido dinámico con un sistema de expresiones evaluadas en tiempo real en el propio navegador.

AngularJS está basado en un sistema de inyección de dependencias donde nuestros controladores piden los objetos que necesitan para trabajar a través del constructor. Luego AngularJS los inyecta de forma tal que el controlador puede usarlo como sea necesario. De esta forma el controlador no necesita saber cómo funciona la dependencia ni cuáles son las acciones que realiza para entregar los resultados.

5. EmbedJS 4.2.1

EmbedJS es un framework Javascript multiplataforma para dispositivos embebidos (móviles, televisiones, etc.). Embed es un proyecto de Dojo Foundation y es un software libre. La gran ventaja que distingue a EmbedJS respecto a otros framework del lado cliente es que EmbedJS simplemente utiliza el código que necesita para cada dispositivo. Esto significa que habrá menos código que se debe cargar en el dispositivo y por tanto habrá menos código viajando en la ejecución de la aplicación y menos uso de memoria.

EmbedJS hace uso de un plugin para decidir que implementación debe ser usada a la hora de ejecutar las características que se requieren en cada proyecto; por tanto, a la hora de realizar un proyecto, simplemente se hará uso de las características que se requieran sin necesidad de hacer uso de código que nunca será utilizado.

6. BackboneJS 1.3.3

BackboneJS es una librería Javascript libre y muy ligera, que se basa también en el patrón MVC (Model-View-Controller). Backbone complementa la estructura de las aplicaciones web añadiéndolas eventos personalizables, colecciones de APIs con numerosas funciones, vistas mediante el manejo de eventos, y todo lo conecta con la API existente mediante una interfaz RESTful.

Cuando se trabaja en aplicaciones web que tienen mucho Javascript, se termina teniendo una aplicación llena de selectores y peticiones para mantener los datos sincronizados, pero con Backbone los datos se presentan como modelos que pueden ser creados, validados, destruidos y guardados en el servidor. Siempre que una acción UI cause el cambio de un modelo, dicho modelo disparará un evento de cambio, y todas las vistas en las que se visualiza dicho modelo serán notificadas del cambio, siendo capaces de responder actualizándose ellas mismas con la nueva información. Es decir, la sincronización y la gestión de eventos son más simples y eficaces.

3.13 Tipos de aplicaciones web

Después de analizar las principales tecnologías del lado del cliente se mostrará en los siguientes apartados las características de cada uno de los tipos de aplicación web, dependiendo de la funcionalidad que se provee en el lado del cliente. La información no pretende ser exhaustiva ni cubrir todas las posibilidades y características, si no dar una idea general y fijar los conceptos vistos.

1. Página web estática

Consiste en la siguiente secuencia de acciones:

- El navegador hace petición al servidor mediante HTTP
- El servidor transforma URL a ruta en disco
- El servidor devuelve el fichero de disco al navegador
- El navegador visualiza (renderiza) la página HTML con estilos CSS e imágenes (sin Javascript).
- Cuando el usuario hace clic en un enlace, el navegador repite el proceso con la URL del link y recarga por completo la página web

Con esta arquitectura el servidor siempre devuelve los mismos recursos, por este motivo, desde el punto de vista del servidor, la web es estática ya que no cambia.

Este tipo de web está formada por HTML, CSS, imágenes, PDF, etc... (pero no incluye Javascript) y al principio de la WWW todas las páginas web eran así (no existía el concepto de aplicaciones web).

Actualmente esta arquitectura se usa principalmente para páginas personales, páginas de proyectos software o documentación técnica.

2. Página web interactiva

Al igual que en el caso anterior, el contenido de la página web está alojado en el disco duro del servidor (estático), pero el cliente es dinámico porque las páginas incluyen código Javascript que se ejecuta en el navegador y puede usarse para incluir efectos gráficos:

- Efectos gráficos que no se pueden implementar con CSS
- Mostrar u ocultar información en función de los elementos que se seleccionan (para documentos largos)
- Menús desplegables
- Páginas adaptables para móviles (responsive)

En este caso, cuando el servidor web recibe una petición, dependiendo de la URL podrá decidir o bien devolver contenido del disco, o bien ejecutar código para generar el recurso dinámicamente. Cuando se ejecuta código, normalmente se hacen consultas a una base de datos para recuperar la información.

La página HTML se genera de forma dinámica pero también puede generar imágenes, PDFs, etc... y si el usuario pulsa un link, se recarga la página al completo.

Esta es la arquitectura de las primeras aplicaciones web pero todavía sigue habiendo muchas páginas web con esta arquitectura.

El contenido es dinámico, porque se ejecuta código en el servidor para generar dicho contenido, y aunque la experiencia de usuario antes no era muy buena por las conexiones lentas y las recargas, ahora han mejorado con la mayor velocidad de Internet y gracias a que ahora los navegadores muestran la página una vez cargada.

La mayoría de las aplicaciones web actuales son dinámicas tanto en cliente como en servidor, y dependiendo de cómo se use el Javascript en el cliente se diferencian tres arquitecturas que veremos a continuación: aplicación web interactiva, aplicación web con AJAX y aplicación web SPA.

3. Aplicación web interactiva

En estas aplicaciones web el Javascript se utiliza para crear efectos gráficos ya que Javascript se diseñó, entre otras cosas, para añadir efectos gráficos básicos a las páginas cuando el CSS era muy limitado.

El dinamismo en el cliente se utiliza exactamente igual que en las páginas web interactivas, son el tipo más básico aunque la gran mayoría de las aplicaciones web que existen en Internet siguen esta arquitectura.

4. Aplicación web con AJAX

Como características de estas aplicaciones web, Javascript se usa para no tener que recargar completamente la página al pulsar un link, de forma que permite hacer petición al servidor web en segundo plano (oculta al usuario). Es decir, cuando llega al navegador el resultado de la petición, el código Javascript actualiza aquellas partes de la página necesarias. A esta técnica se la conoce como AJAX (Asynchronous Javascript And XML).

Usar AJAX en una página mejora mucho la experiencia de usuario ya que no es necesario recargar la página al completo, sólo aquellas partes que cambian y la página se puede cargar por partes, primero la información importante y en segundo plano otros elementos complementarios. Además, se puede dar realimentación al usuario de formas más adecuadas (cuadro de diálogo, error de validación en un formulario, quitar el icono de carga de un recurso, etc...).

5. Aplicación web SPA

El acrónimo SPA (Single Page Application) consiste en llevar la técnica AJAX al extremo y que todo el contenido dinámico se cargue con Javascript en segundo plano, de tal forma que existe una única página cuyo contenido va cambiando según el usuario interactúa con botones, pestañas, etc.

Además, el botón de atrás del navegador funciona porque se “emula” una navegación por páginas cuando se evoluciona por los estados de la aplicación, lo que conlleva en ocasiones una enorme complejidad. La gran popularidad de este tipo de aplicaciones ha venido de la mano de grandes empresas de internet como Google con Gmail y Maps.

4 UX. Experiencia de usuario.

La experiencia de usuario o en inglés user experience (UX) es el conjunto de factores y elementos relativos a la interacción del usuario, con un entorno o dispositivo concretos, cuyo resultado es la generación de una percepción positiva o negativa de dicho servicio, producto o dispositivo. Ésta depende no sólo de los factores relativos al diseño (hardware, software, usabilidad, diseño de interacción, accesibilidad, diseño gráfico y visual, calidad de los contenidos, etc) sino además de aspectos relativos a las emociones, sentimientos, construcción y transmisión de la marca, o la confiabilidad del producto.

La experiencia de usuario como disciplina se aplica tradicionalmente a los sistemas informáticos y en particular al diseño de aplicaciones web, pero hoy se ve ampliada a otros campos. Así pues, ya que se considera como una experiencia casi cualquier producto o servicio, éstos comienzan a diseñarse buscando la máxima satisfacción del consumidor ahora visto como un usuario que transita por estas experiencias.

4.1 Especificaciones funcionales

Las especificaciones funcionales son un paso que se debe realizar para facilitar las experiencias del usuario. Se debe prestar especial atención a la descripción detallada de las funcionalidades que el sitio debe incluir para satisfacer las necesidades del usuario. Requiere además de una descripción del contenido que se requerirá en el sitio, es decir, qué tipo de información se manejará, para quién estará disponible.

Uno de los problemas del funcionamiento de las páginas web es que los usuarios en ocasiones utilizan las funcionalidades por su cuenta de formas distintas a la intención original del diseñador web. Esto se debe a que para la gran mayoría de los usuarios carece de importancia el llegar a entender el funcionamiento de las cosas mientras consigan usarlas para alcanzar su objetivo.

En una aplicación web los componentes deben funcionar en su conjunto, logrando que el usuario reconozca de manera instantánea su utilidad sin distraerle de sus intereses iniciales y que tampoco requiera demasiado tiempo o energía pensar cómo funciona, pues si no se tiene en cuenta este punto el usuario pasará más tiempo descifrando cómo funciona lo que se muestra en pantalla y le dedicará menos tiempo a lo que realmente buscan, convirtiendo la aplicación web en algo ineficaz, así que lo mejor es crear lo más claro posibles los contenidos con el fin de que en una simple ojeada se detecten los elementos más importantes.

Es importante no perder de vista que realizar una especificación funcional no es lo mismo que detallar una especificación técnica. La diferencia radica en que una especificación funcional describe cómo funcionará un producto completamente desde la perspectiva del usuario. No le importa cómo se implemente. Habla de funciones. Especifica pantallas, menús, diálogos, etcétera. Por otro lado, la especificación técnica describe la implementación interna del programa. Habla de estructuras de datos, modelos de bases de datos relacionales, elección de lenguajes y herramientas de programación o algoritmos.

4.2 Diseño de interacción

Es el proceso de generar y evaluar productos, sistemas o dispositivos que cumplan con el objetivo para el que fueron diseñados, y que apoyen las actividades que la gente realiza en su trabajo y en su vida diaria. De manera que es la técnica por la cual se producen interfaces adecuadas entre el ser humano y los sistemas automatizados, con el fin de disminuir la curva de aprendizaje y eliminar barreras en la frontera humano-máquina.

El diseño de interacción tiene como objetivo presentar a los usuarios de las aplicaciones web experiencias útiles, satisfactorias y usables. El diseño de interacción implica actividades como son: el desarrollo de bocetos, la construcción de prototipos y la evaluación de los resultados. Mediante la realización de estas actividades el diseño de interacción investiga, propone y evalúa el desempeño de las aplicaciones web en relación con sus usuarios.

Por otro lado, quien lleva a cabo el diseño de interacción es un profesional que trabaja para definir cómo tiene que funcionar el producto para que sea útil y usable. Para conseguirlo, debe diseñar el producto desde el punto de vista del usuario, no desde el punto de vista técnico ni artístico. El diseñador de interacción debe interiorizar y entender los objetivos, tareas, necesidades y deseos que los usuarios tienen cuando están usando el producto.

4.3 Arquitectura de la información

Dada la ingente cantidad de datos e información que se genera continuamente y a la que deben dar acceso las aplicaciones web, se requieren de una estructura y un desarrollo de métodos, disciplinas y a la vez de profesionales que permitan focalizar la necesidad de información de los usuarios, puesto que Internet ofrece diversas aplicaciones y alternativas de desarrollo para quien lo usa y ha contribuido a la formación de nuevas actividades, como la especializada en la Arquitectura de la Información (AI).

Podemos definir al arquitecto de la información como el encargado de diseñar y crear el edificio, sus componentes, sus pasillos, sus espacios abiertos y sus zonas de servicios. Sólo que ahora el edificio es digital, sus componentes son los contenidos informativos, y el espacio interno del mismo se crea con la organización de los contenidos y con los sistemas de navegación, acceso y etiquetado de la información.

4.4 Diseño de información

El diseño de información puede definirse como el arte y la ciencia de preparar la información, de modo que pueda usarse por los humanos con eficacia y eficiencia. Los autores identifican estos tres requisitos de un buen diseño de información: documentos comprensibles, sistemas interactivos y espacios de información navegables.

El diseño de información para las aplicaciones web es una disciplina en auge y en constante desarrollo. La novedad del medio, unida a su interactividad y a la aparición de nuevas técnicas y herramientas, ha permitido constituir un campo nuevo en el que se han integrado progresivamente profesionales y expertos provenientes de muy diversas áreas, cada uno de los cuales ha aportado aproximaciones y métodos que lo han enriquecido. El proceso de creación de documentos digitales es inherente al desarrollo actual de las ciencias de la información y la documentación, por lo que en este marco debe entenderse el diseño de información.

Diseñar información es trabajar con datos que pueden llegar a convertirse en información valiosa para un lector. En el diseño se persigue no sólo la comunicación de información, sino también su asimilación o aprendizaje a fin de convertirse en conocimiento. En el diseño de sistemas informativos, se optimizan recursos con los que se cuenta, como los humanos, los perceptuales -visuales, auditivos, táctiles, etc.-, los económicos, entre otros. La información que se diseña se hace con base en la intención, experiencia, cultura, maneras de ver y sentir, y otras características propias del lector o usuario.

4.5 Diseño de navegación

La estructura de navegación por una aplicación web viene definida por la experiencia de navegación del visitante en virtud de la cual puede saltar de una página a otra dentro del sitio web utilizando el sistema de hipervínculos. La estructura de navegación se suele elegir en función del tipo de contenido y existen distintos tipos de estructuras:

1. **Jerárquica:** es una estructura en árbol donde existe una página índice o principal desde donde se accede al resto de páginas. Desde estas subpáginas se puede acceder a otras y así sucesivamente creando distintos niveles o jerarquías. Es ideal para sitios web de centros o proyectos. No se aconseja utilizar más de 4 niveles para evitar que el usuario se desoriente durante la navegación. Conviene situar en todas las páginas un menú que permita moverse de una forma fácil y directa por los distintos niveles y páginas de cada nivel.
2. **Lineal:** es una estructura muy simple similar a las páginas de un libro. Desde una página concreta se puede ir a la página siguiente o la página anterior. Es especialmente útil si deseamos

que el usuario siga un itinerario fijo y guiado sin posibilidad de acceder a otras páginas que pudieran distraerle. Ejemplo: Guía o tutorial de aprendizaje. No es recomendable si el número de páginas encadenadas es muy elevado porque produce sensación de fatiga y no permite retomar fácilmente la secuencia allí donde se abandonó en la última sesión.

3. **Lineal con jerarquía:** es una estructura híbrida que trata de aprovechar las ventajas de las dos estructuras anteriores. Las páginas y subpáginas se organizan de forma jerárquica pero también es posible navegar de forma lineal por las páginas de un mismo nivel.
4. **Red:** a partir de la página índice o principal se puede navegar a otra u otras sin ningún orden aparente. Es una estructura más libre pero no es aconsejable cuando el número de páginas es elevado porque desorienta al usuario al no saber dónde está ni disponer de recursos para ir donde desea.

4.6 Diseño visual aplicado a la web

Una manera simple de evaluar el diseño visual de una aplicación web es preguntar a un usuario: ¿Adónde se dirige la mirada primero? ¿Qué elementos llaman la atención del usuario inicialmente? ¿Son estos los elementos más importantes en los objetivos estratégicos? Tengamos en cuenta que los aspectos visuales pueden influir psicológicamente en un usuario, y que el diseñador debe tener en cuenta esto y tratar de influenciar al usuario para que realice sus objetivos de forma eficiente y sencilla. Si la aplicación web que nos encontramos desarrollando se tratase de una tienda virtual, el visitante no sólo deberá sentirse atraído por los productos ofertados, sino que además deberá desear acceder a uno de ellos.

Los diseños visuales en las aplicaciones web en la actualidad se encuentran conformados por una serie de bloques o módulos dispuestos en una forma más o menos estandarizada, que permiten a los usuarios captar con simples recorridos visuales donde se encuentran los diferentes elementos que componen una página. Esta disposición estandarizada, responde a modelos de lógica visual que están relacionados con el comportamiento de los usuarios.

Sobre la base de esto, los diseñadores gráficos deben plantearse como objetivos la creación de una jerarquía visual que permita al usuario de una página web detectar de inmediato que elementos son importantes y que otros son secundarios, definir cuáles serán las regiones funcionales del sitio y formar grupos de elementos relacionados de forma tal que los visitantes puedan ver claramente la estructura del contenido.

En el diseño de páginas web, deben emplearse modelos que a los usuarios les resulten familiares, lo que colabora en la ubicación por parte de los mismos de las áreas funcionales y los contenidos importantes. Los diseños excesivamente llenos de elementos, con fondos cargados o con bajos contrastes, pueden dificultar el establecimiento de puntos de referencia visual que hagan comprensible la organización del contenido.

Al navegar por diferentes páginas de una aplicación web, el usuario debe tener la sensación de que las diferentes páginas que visitan pertenecen a una misma aplicación. Esto se logra mediante la repetición de algunos elementos, que dan un estilo definido a la aplicación. Esta repetición, además de dar una unidad visual a toda la aplicación, permite a los visitantes ubicar los elementos más importantes, ya que se encontrarán con la misma distribución en todos los puntos de esta aplicación.

4.7 Usabilidad

Relacionado con la experiencia de usuario que hemos tratado en el apartado anterior, tenemos la usabilidad. El término usabilidad se refiere a la facilidad con que las personas pueden utilizar una herramienta particular o cualquier otro objeto fabricado por humanos con el fin de alcanzar un objetivo concreto, en el caso de este tema las aplicaciones web. La usabilidad también puede referirse al estudio de los principios que hay tras la eficacia percibida de un objeto. Aunque no hay consenso sobre las diferencias entre usabilidad y experiencia de usuario está comúnmente aceptado que la usabilidad está más orientada a la eficiencia en el uso de unas aplicaciones para alcanzar los objetivos, mientras que la UX está más orientada a la percepción de satisfacción del usuario (algo más subjetivo), aunque a menudo ambas coinciden ya que los usuarios disfrutan de la experiencia cuando consiguen sus objetivos de forma eficaz.

El grado de usabilidad de un sistema es una medida empírica y relativa de la usabilidad del mismo. Se mide a partir de pruebas empíricas y relativas. Se considera empírica porque no se basa en opiniones o sensaciones, sino en pruebas de usabilidad realizadas en laboratorio u observadas mediante trabajo de campo. Y también se considera relativa, porque el resultado no es ni bueno ni malo, sino que depende de las metas planteadas o de una comparación con otros sistemas similares.

Los principios básicos en los que se basa la usabilidad son:

- **Facilidad de aprendizaje:** facilidad con la que nuevos usuarios desarrollan una interacción efectiva con el sistema o producto. Está relacionada con la predictibilidad, sintetización, familiaridad, la generalización de los conocimientos previos y la consistencia.
- **Facilidad de uso:** facilidad con la que el usuario hace uso de la herramienta, con menos pasos o más naturales a su formación específica. Tiene que ver con la eficacia y eficiencia de la herramienta.
- **Flexibilidad:** relativa a la variedad de posibilidades con las que el usuario y el sistema pueden intercambiar información. También abarca la posibilidad de diálogo, la multiplicidad de vías para realizar la tarea, similitud con tareas anteriores y la optimización entre el usuario y el sistema.
- **Robustez:** es el nivel de apoyo al usuario que facilita el cumplimiento de sus objetivos. Está relacionada con la capacidad de observación del usuario, de recuperación de información y de ajuste de la tarea al usuario.

En informática, la usabilidad está muy relacionada también con la accesibilidad, hasta el punto de que algunos expertos consideran que una forma parte de la otra o viceversa.

4.8 Usabilidad web

La usabilidad web es la disciplina que estudia la forma de diseñar sitios web para que los usuarios puedan interactuar con ellos de la forma más fácil, cómoda e intuitiva posible. La mejor forma de crear un sitio web usable es realizando un diseño centrado en el usuario, es decir, diseñando para y por el usuario, en contraposición a lo que podría ser un diseño centrado en la tecnología o uno centrado en la creatividad u originalidad.

Una parte fundamental de la usabilidad de un sitio web está determinada por los signos gráficos que representan elementos de control y medida para los usuarios y les proporcionan referentes visuales relacionados a elementos que se encuentran en su vida cotidiana y que al interactuar con ellos en el sitio web les permite con facilidad comprenderlo y utilizarlo. Los signos gráficos pueden ser imágenes, botones o menús de navegación.

La investigación en usabilidad web se centra en la comodidad, navegación, arquitectura de la información y contenido de una aplicación web, y es importante tener en cuenta que la percepción de usabilidad tiene una influencia positiva en el usuario respecto a la confianza, satisfacción y lealtad hacia un sitio web.

Existen 5 reglas principales que adaptadas a una web, se les puede considerar como una web "usable".

1. **Rápido** - Las páginas deben cargarse en una media de 4 segundos. Los usuarios lo más que esperarán en ver el contenido de una página web es de una media de 10 segundos. - La mayoría de los usuarios disponen de módem para su acceso a Internet, por lo que nuestras páginas deben de ser lo menos pesadas posibles con el fin de que los usuarios no esperen mucho tiempo, porque de lo contrario cancelarán la visita.
2. **Simple** - Mantenga una navegación constante. No fuerce a los visitantes a aprender diversos caminos o esquemas para la navegación en diversas partes de su site. - No abuse de la utilización de la animación, esto puede abrumar y cansar la vista.
3. **Investigable** - Los motores de búsqueda buscan el texto real. No prestan ninguna atención a los gráficos y al código de programación (como el Javascript). Evite estas situaciones si desea que su web esté bien posicionada en los buscadores.

4. **Para la mayoría** - Los sitios web necesitan ser compatibles con todos los navegadores y ordenadores para su fácil usabilidad. - Utilice HTML simple y llano siempre que sea posible, es el código más compatible con todos los navegadores.
5. **Manténgalo actualizado** - La manera más rápida para que una web pierda credibilidad es contener la información anticuada.

Actualmente las personas tienen mayor acceso a contenidos web desde dispositivos móviles como los teléfonos inteligentes, que desde computadoras de escritorio, ya que esto les permite estar comunicados y disponibles en cualquier momento y en cualquier situación que lo requieran; sin embargo, cuando el usuario navega en la web móvil, se enfrenta a problemas de usabilidad debido a que muchos de los contenidos web no están adaptados para visualizarse en dispositivos con pantallas pequeñas y aquellos que sí lo están, inciden en problemas de navegación, presentación de la información, cantidad adecuada de información, uso adecuado de los gráficos, etc.

Con el fin de mejorar el diseño de los interfaces móviles de las aplicaciones web han surgido los enfoques del responsive web design y del adaptative web design, ambas persiguen el mismo objetivo que es servir a cada dispositivo un interfaz adaptado a las dimensiones de su pantalla que mejore la interacción. La diferencia entre ambas está en que el responsive design propone un enfoque continuo de ajuste de interfaz, mientras que el adaptative design define rangos discretos de tamaño de pantalla para cada una de las versiones del diseño.

Para mejorar la experiencia de usuario en servicios móviles es necesario considerar dos aspectos importantes de la usabilidad: la simplicidad y la interactividad. La simplicidad es considerada un concepto crucial para el diseño exitoso de una interfaz de usuario porque la creciente complejidad de la tecnología hace nuestra vida compleja y desordenada. La interactividad ha sido estudiada como una característica esencial de los sitios web que conduce las actividades y actitudes de los usuarios.

Para finalizar este apartado se van a indicar algunos de los principales beneficios de la usabilidad:

- Reducción de los costes de aprendizaje y esfuerzos.
- Disminución de los costes de asistencia y ayuda al usuario.
- Disminución en la tasa de errores cometidos por el usuario y del trabajo innecesario.
- Optimización de los costes de diseño, rediseño y mantenimiento.
- Aumento de la tasa de conversión de visitantes a clientes de un sitio web.
- Aumento de la satisfacción y comodidad del usuario.
- Mejora la imagen y el prestigio.
- Mejora la calidad de vida de los usuarios, ya que reduce su estrés, incrementa la satisfacción y la productividad.

Todos estos beneficios implican una reducción y optimización general de los costes de producción, así como un aumento en la productividad. La usabilidad permite mayor rapidez en la realización de tareas y reduce las pérdidas de tiempo.

5 Desarrollo web en servidor

El extremo del servidor es la parte más compleja y pesada de la arquitectura como ya se ha mencionado. Este extremo debe ser capaz de atender las demandas variables de los usuarios, y debe hacerlo en tiempos que sean adecuados para poder trabajar. Esta capacidad de adaptación a la demanda requiere de características como la escalabilidad, el balanceo de carga, el cacheo de información estática, y otras muchas técnicas que permitan ofrecer un servicio de calidad y al mismo tiempo realizar un uso de los recursos eficiente.

5.1 Lenguajes y tecnologías

Se ofrecen a continuación un listado con algunos datos sobre las tecnologías web del lado servidor que más se están usando en la actualidad. Se analizan sin bajar a profundidad de detalles ya que como se ha mencionado el objetivo es dar una panorámica global, de tal forma que para ampliar la información de cada tecnología se recomienda acudir a los temas específicos que trata cada una.

5.1.1 PHP 7.1.9

PHP (acrónimo recursivo de PHP: Hypertext Preprocessor) es un lenguaje de código abierto muy popular especialmente adecuado para el desarrollo web y que puede ser incrustado en HTML. Las páginas de PHP contienen HTML con código incrustado que está encerrado entre las etiquetas especiales de comienzo y final `<?php` y `?>` que permiten entrar y salir del "modo PHP".

La diferencia de PHP con Javascript, es que el código es ejecutado en el servidor, generando HTML y enviándolo al cliente. El cliente recibirá el resultado de ejecutar el script, aunque no leerá el código subyacente que lo generó. El servidor web puede ser incluso configurado para que procese todos los ficheros HTML con PHP, por lo que no hay manera de que los usuarios puedan saber qué contienen los ficheros originales.

PHP está enfocado principalmente a la programación de scripts del lado del servidor, por lo que se puede recopilar datos de formularios, generar páginas con contenidos dinámicos, o enviar y recibir cookies. Con PHP se tiene la libertad de elegir el sistema operativo, ya que puede usarse en todos los principales. Además, se tiene la posibilidad de utilizar programación por procedimientos o programación orientada a objetos (POO), o una mezcla de ambas.

Entre las capacidades de PHP se incluyen la creación de imágenes, ficheros PDF e incluso películas Flash (usando libswf y Ming). También se puede generar fácilmente cualquier tipo de texto, como XHTML y cualquier otro tipo de fichero XML. PHP puede autogenerar estos ficheros y guardarlos en el sistema de ficheros en vez de imprimirlos en pantalla, creando una caché en el lado del servidor para contenido dinámico.

Una de las características más potentes y destacables de PHP es su soporte para un amplio abanico de bases de datos. También cuenta con soporte para comunicarse con otros servicios usando protocolos tales como LDAP, IMAP, SNMP, NNTP, POP3, HTTP, COM (en Windows) y muchos otros.

5.1.2 Java EE 8

Esta tecnología ha recibido muchos nombres como Java Platform, Enterprise Edition o Java EE (anteriormente conocido como Java 2 Platform, Enterprise Edition o J2EE hasta la versión 1.4; traducido informalmente como Java Empresarial). Es una plataforma de programación para desarrollar y ejecutar software de aplicaciones en el lenguaje de programación Java.

Permite utilizar arquitecturas de N capas distribuidas y se apoya en componentes de software modulares ejecutándose sobre un servidor de aplicaciones. La plataforma Java EE está definida por una especificación del Java Community Process, y Java EE es también considerado informalmente como un estándar debido a que los proveedores deben cumplir ciertos requisitos de conformidad para declarar que sus productos son conformes a Java EE; estandarizado por The Java Community Process / JCP.

Java EE tiene varias especificaciones de API, tales como JDBC, RMI, e-mail, JMS, servicios web, XML, etc y define cómo coordinarlos. Java EE también configura algunas especificaciones únicas para Java EE para componentes. Estas incluyen Enterprise JavaBeans, servlets, portlets (siguiendo la especificación de Portlets Java), JavaServer Pages y varias tecnologías de servicios web, algunas de las cuales se verán más adelante en el tema. Ello permite al desarrollador crear una Aplicación de Empresa portable entre plataformas y escalable, a la vez que integrable con tecnologías anteriores.

Otros beneficios son, que el servidor de aplicaciones puede manejar transacciones, la seguridad, escalabilidad, concurrencia y gestión de los componentes desplegados, significando que los desarrolladores pueden concentrarse más en la lógica de negocio de los componentes en lugar de en tareas de mantenimiento de bajo nivel.

5.1.3 ASP 3.0

ASP (Active Server Pages), al igual que el PHP, es un lenguaje del lado del servidor que permite hacer páginas dinámicas con accesos a bases de datos, conexiones de red, envío de mails, etc. Este tipo de tecnologías tiene como ventaja la seguridad que le dan al programador: es él quien tiene acceso al código, ya que el mismo se encuentra solo en los servidores.

ASP es el lenguaje estándar que creó Microsoft para su servidor web IIS, por lo tanto solo funciona en plataformas de esa firma (Windows), aunque existen algunas iniciativas como mod-mono para Apache que nos permite utilizar esta tecnología en otros sistemas operativos.

Lo interesante de este modelo tecnológico es poder utilizar diversos componentes ya desarrollados como algunos controles ActiveX así como componentes del lado del servidor, tales como CDONTS, por ejemplo, que permite la interacción de los scripts con el servidor SMTP que integra IIS.

5.1.4 ASP.NET 5.0

ASP.NET es un entorno para aplicaciones web desarrollado y comercializado por Microsoft. Es usado por programadores y diseñadores para construir sitios web dinámicos, aplicaciones web y servicios web XML. Es la tecnología sucesora de la tecnología Active Server Pages (ASP) vista antes. ASP.NET está construido sobre el Common Language Runtime, permitiendo a los programadores escribir código ASP.NET usando cualquier lenguaje admitido por el .NET Framework.

ASP.NET permite escribir código del lado del servidor usando diferentes tipos de lenguajes. El Framework .NET actualmente soporta alrededor de 40 lenguajes y muchos de éstos pueden ser usados para construir aplicaciones ASP.NET. C# es una de las elecciones más populares.

Los scripts de ASP.NET son compilados en la primera ejecución, Just In Time (JIT). ASP.NET tiene acceso completo a las funcionalidades del Framework .NET incluyendo soporte para XML, servicios web, interacción con bases de datos, correo, etc. ASP.NET trae consigo una estructura que separa el código que contiene la lógica de las páginas HTML.

ASP.NET es un Framework que ya incluye diferentes controles y librerías disponibles para el desarrollador. Para construir ágilmente aplicaciones empresariales se requiere usar Visual Studio, ya sea en sus versiones libres (Express) o de pago.

Las aplicaciones se ejecutan sobre el servidor de Internet de Microsoft IIS, permitiendo la conectividad con bases de datos (ODBC), y la gestión de concurrencia a nivel de base de datos. El entorno también realiza la gestión de transacciones para bases de datos y servicios web con MTS.

5.1.5 Ruby 2.4.1

Ruby es un lenguaje de programación interpretado, reflexivo y orientado a objetos. Combina una sintaxis inspirada en Python y Perl con características de programación orientada a objetos similares a Smalltalk. Comparte también funcionalidad con otros lenguajes de programación como Lisp, Lua, Dylan y CLU. Ruby es un lenguaje de programación interpretado en una sola pasada que está diseñado para la productividad y la diversión del desarrollador, siguiendo los principios de una buena interfaz de usuario.

Ruby está orientado a objetos, y todos los tipos de datos son un objeto, incluidas las clases y tipos que otros lenguajes definen como primitivos. Toda función es un método. Las variables siempre son referencias a objetos, no los objetos mismos. Ruby soporta herencia con enlace dinámico, mixins y métodos singleton (pertenecientes y definidos por una sola instancia más que definidos por la clase). A pesar de que Ruby no soporta herencia múltiple, las clases pueden importar módulos como mixins. Es compatible con la sintaxis procedimental, pero todos los métodos definidos fuera del ámbito de un objeto son realmente métodos de la clase `Object`. Como esta clase es padre de todas las demás, los cambios son visibles para todas las clases y objetos.

Ruby es un lenguaje de programación multiparadigma ya que permite programación procedimental, con orientación a objetos, (todo es un objeto) o funcionalmente. Ruby tiene tipado dinámico, y soporta polimorfismo de tipos (permite tratar a subclases utilizando la interfaz de la clase padre). Ruby no requiere de polimorfismo de funciones (sobrecarga de funciones) al ser dinámicamente tipado (los parámetros de una función pueden ser de distinto tipo en cada llamada, encargándose dicha función de determinar el tipo y actuar en consecuencia).

Además del lenguaje básico, existen distintas implementaciones de Ruby que extienden sus capacidades, como JRuby, que permite a Ruby utilizar la máquina virtual de Java o MacRuby, que permite integrar Ruby con las librerías Cocoa de Apple, para el desarrollo de aplicaciones sobre esta plataforma.

5.1.6 Python 3.6.2

Python es un lenguaje de alto nivel de propósito general y sus características principales son: orientación multiparadigma, soporta orientación a objetos, programación estructurada, programación funcional y orientación a aspectos, tipado dinámico, sintaxis escueta, delimitación de bloques mediante indentación (en lugar de llaves o paréntesis), alta extensibilidad mediante un núcleo básico reducido y una gran cantidad de librerías estándar, recolector de basura automático y resolución de nombres tardía (se ligan los nombres de métodos y variables durante la ejecución del programa).

Entre todos los usos posibles del lenguaje tenemos: el de desarrollo de aplicaciones web, el acceso a bases de datos, la creación de interfaces de usuario de escritorio, el cálculo científico y numérico, o el desarrollo de software y de videojuegos.

Python es utilizado por grandes empresas como Google, Yahoo u organizaciones como la NASA y el CERN, en algunos de sus proyectos. Mediante el uso de librerías como Matplotlib, NumPy y SciPy, Python está ganando muchos adeptos en el ámbito de la computación científica, incluido el análisis estadístico y matricial. Además, es un componente estándar de muchas distribuciones Linux, muchas de las cuales usan incluso instaladores basados en él. También es el lenguaje principal del microordenador Raspberry Pi y LibreOffice ha portado todo su código de Java a Python.

5.2 Frameworks Back-end

En el siguiente epígrafe se recogen algunos de los frameworks más importantes de las tecnologías web de servidor que se han descrito antes. No cubren todos los posibles frameworks pero sí los más populares cuando se redactó este tema, por lo que se recomienda al opositor que revise si hay algún otro que haya irrumpido nuevo y se deba también conocer, o alguno de los que se mencionan que haya perdido importancia.

5.2.1 Frameworks Java EE

1. Spring 4.3.10

Spring es un framework multicapa de aplicaciones de Java EE, que incluye un contenedor ligero para la configuración y conexión automáticos de objetos de aplicación mediante inyección de dependencias, una capa de abstracción para la gestión de transacciones, una capa de abstracción JDBC (Java DataBase Connectivity), funcionalidad de programación orientada a aspectos e integración con mapeadores objeto-relacional (ORM), configuración mediante JMX, una mejora de la API de envío de mensajes JMS, soporte para tests unitarios, su propio framework MVC y soporte para la construcción rápida de aplicaciones, entre otros módulos. Sus componentes básicos pueden

ser utilizados para cualquier aplicación Java, pero existen módulos de expansión que permiten construir aplicaciones web sobre Java EE utilizándolo.

2. Apache Tapestry 5

Tapestry es un framework de aplicaciones de Java creado como un proyecto de la Fundación Apache. Sus características principales son: estructura estática y comportamiento dinámico, ofrecer una API adaptativa y que se diferencie entre la API interna y pública, y asegurar retrocompatibilidad.

El framework prioriza la simplicidad, sencillez de uso y productividad del programador. Además ofrece una convención sobre la configuración, por lo que elimina prácticamente toda la configuración en XML. Dispone de una aproximación modular al desarrollo web, mediante el uso de asociaciones entre cada componente de la interfaz de usuario y su clase Java correspondiente.

3. Spark framework 2.6

Spark es un nuevo tipo de framework que ha ganado popularidad en los últimos años, ya que es ligero, sencillo y está orientado al despliegue de aplicaciones lo más rápidamente posible. No necesita Java EE, únicamente Java y provee una API limitada pero muy focalizada basada en tres pilares:

- **Objetos Request y Response:** permiten controlar las cabeceras HTTP y sus funcionalidades. Equivalente a la API de Servlets.
- **Routes:** los puntos de unión entre la aplicación y el protocolo HTTP. Permite establecer funciones de callback a realizar para cada método HTTP específico.
- **Filters:** pequeños métodos que pueden ser insertados antes o después de la ejecución de una petición y permiten ser activados únicamente en ciertas rutas.

Toda la configuración de la aplicación se realiza mediante código Java, no existiendo configuraciones XML o anotaciones.

4. Apache Struts 2.5.12

Struts es una herramienta para el desarrollo de aplicaciones web que sigue el patrón MVC sobre la plataforma Java EE. Struts fue desarrollado como parte del proyecto Jakarta de la Apache Software Foundation, pero actualmente es un proyecto conocido como Apache Struts. Struts permite reducir el tiempo de desarrollo. Struts se basa en el patrón de arquitectura de software Modelo-Vista-Controlador (MVC) el cual se utiliza ampliamente y es considerado de gran solidez.

5. Maverick 2

Maverick es un framework para desarrollo web que utiliza Java y Java EE basado en el Modelo-Vista-Controlador (MVC). Es un framework minimalista que se centra en la lógica MVC, permitiendo generar la presentación utilizando una amplia variedad de plantillas y lenguajes de transformación.

Maverick es un framework minimalista y cualquiera puede entenderlo fácilmente. No es un recetario que provea al programador de todo lo que necesite para desarrollar una aplicación web, puesto que ya existen una gran colección de conectores de bases de datos, servidores de aplicaciones, frameworks de prueba o lenguajes de plantillas. Maverick está diseñado para que sea fácilmente extensible mediante plugins. La idea de un controlador que construya un modelo que se obtenga renderizado por una vista es muy simple, por lo que el framework debería ser igual de simple.

5.2.2 Frameworks de Ruby

1. Ruby on Rails 5.1.3

Ruby on Rails es un framework open source full stack que permite crear aplicaciones que tomen datos del servidor además de interactuar con una base de datos y mostrar plantillas. Promueve el uso de patrones de diseño conocidos, como Convention Over configuration, Don't Repeat Yourself.

Toda la estructura de la aplicación se realiza mediante el modelo Modelo-Vista-Controlador, donde:

- Un modelo es una tabla en la base de datos y un fichero Ruby.
- Un controlador es un componente de Rails que responde a peticiones externas del servidor web a la aplicación determinando qué archivo de vista debe mostrar.

- Una vista es un archivo .erb, que se compila en un HTML en tiempo de ejecución.

Este framework incluye servicios para facilitar la tarea de desarrollo que permite construir una web básica automáticamente, o un servidor web escrito en Ruby.

2. Sinatra 2.0

Sinatra surge como alternativa a otros frameworks similares pero mucho más potentes y destinados a aplicaciones de gran tamaño, como Ruby on Rails, de hecho una de sus grandes características es su sencillez y la simplicidad de uso. Esto permite la elaboración de aplicaciones web en un corto espacio de tiempo, lo que le hace muy adecuado para dar interfaz web a servicios ya existentes. Otra de las características de Sinatra es permitir en pocas líneas de código la creación de servicios.

Sinatra es más simple y no ofrece funciones auxiliares que ayudan a crear formularios, conectar a bases de datos, o cualquiera de las otras funciones que ofrece Rails. Por otro lado, Rails requiere rutas diferenciadas para definir cómo la aplicación web va a responder a las solicitudes, que se conecta a los controladores apropiados y modelos. Sinatra funciona de una forma mucho más simple. Cuando se declara un nuevo "get " o "post" de acción, Sinatra se añade automáticamente a la ruta, y simplemente comenzará a responder a las peticiones que se le formulen.

3. Sequel 5

Sequel permite manejar las bases de datos desarrolladas en Ruby, ya que ofrece seguridad para subprocesos, la agrupación de conexiones DSL, la construcción de consultas SQL y esquemas de tabla. Por otra parte incluye una capa ORM para la asignación de registros a objetos Ruby y manejo de los registros asociados.

También soporta características avanzadas de base de datos, tales como declaraciones preparadas, variables ligadas, procedimientos almacenados, puntos de retorno, aislamiento de la transacción, configuraciones maestro-esclavo... Finalmente, indicar que Sequel cuenta actualmente con adaptadores para ADO, DB2, Informix, JDBC, MySQL, ODBC, Oracle, PostgreSQL, SQLite3.

5.2.3 Frameworks de Javascript

Javascript es un lenguaje pensado originalmente como lenguaje ligero de scripting para la parte de cliente web que se está convirtiendo en el lenguaje para todo en los últimos años, especialmente gracias a Node.js, una de las tecnologías de servidor más utilizadas por su potencia y versatilidad.

1. Node.js 8.4

Node.js es una plataforma construida sobre el motor de Javascript de Chrome para construir fácilmente aplicaciones rápidas y escalables. Node.js utiliza un modelo E/S no bloqueante basado en eventos que lo hacen ligero y eficiente, perfecto para aplicaciones de tiempo real y uso intensivo de datos que se ejecutan en sistemas distribuidos. Node.js es una librería de Javascript para el desarrollo de servidores web y sobre ésta, se han desarrollado numerosos frameworks web.

Para entender mejor las características del node.js a continuación se muestra en la Tabla 1 la comparativa entre un servidor web tradicional desarrollado con PHP y uno implementado con node.js.

Característica principal	Node.js	Apache PHP
Concurrencia	Atiende varias peticiones a la vez con un solo hilo	Crea un nuevo hilo por cada conexión cliente-servidor, lo cual funciona bastante bien cuando hay pocas conexiones; sin embargo crear nuevos hilos es costoso, ya que a partir de 400 conexiones simultáneas aumenta exponencialmente el número de segundos necesarios para atender a cada una de las peticiones.
Inicialización y carga del código	Tiene todo cargado en memoria que lo ha cargado una sola vez antes de	Necesita ejecutar la inicialización y carga en cada petición

	gestionar la petición	
Comunidad	Aun no es muy grande	Bastante más grande comparado con Node.js

Tabla 1 Comparativa nodeJS vs Apache PHP

2. Express 4.15.4

Express es un framework accesible y ligero que permite construir aplicaciones de cualquier tamaño. Entre sus características se incluyen la creación de rutas, parseo del cuerpo de las peticiones, parseo de peticiones URL, manejo de cookies, autenticación básica HTTP y renderizado de plantillas. Además, soporta diversos lenguajes de simplificación como Sass, Less (CSS) o Jade (HTML).

3. Meteor 1.5

Meteor es un framework full stack que utiliza Node.js como back-end pero, en la práctica, trabaja como una plataforma independiente. Una aplicación Meteor es una mezcla de Javascript corriendo en el cliente, Javascript en el servidor dentro de un contenedor Node.js y todo el HTML, CSS y resto de recursos que lo soportan.

Se fundamenta en minimizar el envío de HTML a través de la red, enviando únicamente datos y permitiendo al cliente decidir cómo renderizarlos. Es decir, permite escribir las partes de cliente y servidor en un único lenguaje y utilizar la misma API transparente para acceder a la base de datos desde el cliente y el servidor.

Meteor está formado por dos partes, por un lado una librería de paquetes y módulos necesarios en la aplicación, como webapp y templating, que permiten manejar conexiones HTTP entrantes y crear plantillas que se actualizan ante cualquier cambio de los datos. Por otro lado, Meteor es una herramienta de línea de comandos que permite empaquetar la aplicación, realizando todas las tareas necesarias, como compilar lenguajes como CoffeeScript, minimizar CSS, descargar paquetes necesarios, etc.

5.2.4 Frameworks de PHP

1. Symfony 3.3.2

Symfony, es definido como una librería PHP que realiza dos tareas diferentes: por un lado, proporciona una selección de componentes Symfony2 y algunas librerías de terceros; y por otro lado define una configuración adecuada e incluye una capa que integra todas las diferentes partes (componentes, librerías, etc.).

El objetivo de la plataforma es integrar muchas herramientas independientes con el fin de proporcionar una experiencia coherente al desarrollador. De estos componentes las más importantes son la HttpFoundation, con clases para manejar sesiones y cargar archivos; el Routing, como sistema de enrutado para asignar una URI específica a cierta información acerca de cómo se debe manejar una petición; y finalmente el Templating: librería para utilizar plantillas.

2. Laravel 5.5.0

Este framework está construido utilizando componentes de Symfony, Laravel es la plataforma PHP que dispone de enrutado RESTful y utiliza un ORM (Object Relational Mapping) para imponer restricciones sobre relaciones entre objetos de la base de datos. Además, permite utilizar las plantillas nativas de PHP o el motor ligero Blade, y es apropiado tanto para la creación de pequeñas aplicaciones o de sistemas de producción empresariales, y permite la ejecución de tests unitarios de manera sencilla.

3. Zend Framework 3.0

Zend Framework (ZF) es un framework de código abierto para desarrollar aplicaciones web y con servicios web. Tiene una implementación que usa código 100% orientado a objetos, y en su estructura de componentes, cada componente está construido con una baja dependencia de otros componentes, por lo que es una arquitectura débilmente acoplada que permite a los desarrolladores utilizar los componentes por separado.

Aunque se pueden utilizar de forma individual, los componentes de la biblioteca estándar de Zend Framework conforman un framework de aplicaciones web al combinarse. También, ofrece una implementación MVC, una abstracción de base de datos, y un componente de formularios que implementa la prestación de formularios HTML, validación y filtrado para que los desarrolladores puedan consolidar todas las operaciones usando de una manera sencilla la interfaz orientada a objetos.

4. CodeIgniter 3.1.5

El último framework que veremos sobre PHP es el más ligero de los tres, siguiendo con la convención de añadir un framework rápido para cada lenguaje. No necesita prácticamente configuración y tiende a la simpleza y flexibilidad de código frente a librerías monolíticas.

5.2.5 Frameworks de Python

1. Django 1.11.4

Este framework fue creado especialmente para portales de noticias, y fue desarrollado buscando la rapidez de desarrollo. Posee una gran comunidad muy activa. Su principal punto fuerte es la cantidad de herramientas que lleva incluidas por defecto, entre las que destaca una interfaz de administración. Además de ello, tiene su propio lenguaje de plantillas y posee una gran documentación.

2. Web2py 2.14.6

Web2py es un framework full stack para Python, con retrocompatibilidad garantizada hasta la versión 1.0. Este framework no requiere de instalación ni configuración: el entorno de desarrollo está basado en web y se accede a él a través del navegador. Incluye un servidor web, una base de datos relacional, un entorno de desarrollo integrado, una capa de abstracción para tratar con SQL en tiempo real, múltiples sistemas de autenticación, un sistema de logging de errores, múltiples sistemas de cacheo para estabilidad y jQuery para AJAX y efectos.

Previene gran cantidad de fallos de seguridad, como Cross Site Scripting o Code Injection. Además es compatible con gran cantidad de bases de datos, como SQLite, PostgreSQL, MySQL, MSSQL, FireBird, Oracle, etc. También dispone de una gran cantidad de documentación.

5.3 Motores de Plantillas

Un motor de plantillas se define como una herramienta que permite procesar y generar código HTML dinámico a partir de variables o expresiones como condicionales, bucles, etc. La utilidad de este tipo de herramientas reside en que separan la lógica de la presentación, lo que permite modificar cada pieza de manera independiente. La ventaja de un motor de plantillas es que permite controlar la presentación sin tener que modificar a mano el código HTML, además de poder replicar el mismo diseño en todo el servicio de manera sencilla o poder cambiarlo de forma rápida asociando una nueva plantilla.

5.3.1 Motores de Plantillas Java EE

1. JSP: Java Server Pages 2.3

JSP (JavaServer Pages) es una tecnología incluida en la plataforma Java EE que permite desarrollar contenido web dinámico de manera rápida y sencilla. Es la evolución de los Servlet, que genera todo el contenido HTML desde código escrito en Java. Estas páginas JSP mezclan el código HTML con etiquetas particulares que incluyen sintaxis Java. Otra característica destacable es que puede evitarse la repetición de código a través de la directiva include.

2. Spring Template 4.3.10

String Template es un motor de plantillas para Java (aunque también hay una versión para C# y Python), y es de código abierto. Sus plantillas se diferencian del resto de plantillas habituales en que donde habitualmente entre corchetes angulares u otros delimitadores aparece la dupla <atributo-expresión>. String Template trata todo como texto y después renderiza llamando únicamente a un método: ST.render().

3. JSF: Java Server Faces 2.3

Java Server Faces, es una especificación de Java para construir interfaces de usuario en aplicaciones web. JSF 2 hace uso de Facelets como sistema de plantillas por defecto, aunque se pueden emplear otras tecnologías como por ejemplo XUL. Basándose en el diseño del modelo de un componente UI, Java Server Faces usa archivos XML llamados vistas de plantilla o Facelets views.

JSF se suele usar juntamente con AJAX, y dado que JSF soporta múltiples formatos de salida, los componentes de AJAX pueden ser fácilmente añadidos para enriquecer la interfaz web basada en JSF. La especificación de JSF 2.0 provee soporte “built-in” para AJAX, estandarizando el ciclo de vida de las peticiones de AJAX y permitiendo así el desarrollo de interfaces simples para manejar los eventos de AJAX.

5.3.2 Motores de Plantillas Ruby

1. Haml 5.0.3

Haml (HTML Abstraction Markup Language) es un motor de plantillas que permite expresar la estructura de un documento HTML de manera no repetitiva y sencilla. Se basa en el principio “markup should be beautiful”, proporcionando una sintaxis elegante, fácil de entender haciendo uso de la indentación, eliminando así la dificultad de la apertura y cierre de las etiquetas HTML.

El uso de Haml es bastante simple, las etiquetas HTML se sustituyen por el símbolo de porcentaje seguido del nombre de la etiqueta (%body, %div). Tras esto, se pone el símbolo de igualdad para indicar a Haml que evalúe el código Ruby que aparecerá a la derecha de tal símbolo. Los atributos de las etiquetas HTML se escribirán de la siguiente manera: {:nombreAtributo => “valorAtributo”}. Si la etiqueta posee más de un atributo, estos irán separados por comas.

2. Slim 3.0.8

Slim es un lenguaje de plantillas que tiene por objetivo reducir la sintaxis de HTML. Inicialmente fue concebido para ver cuánto se podía eliminar de una plantilla HTML estándar. Su sintaxis es simple, elegante y soportada por la mayoría de los frameworks de Ruby. Slim utiliza la herramienta Temple para realizar el parseado y la compilación, y también usa la herramienta Tilt, como interfaz genérica de múltiples motores de plantillas. Este motor de plantillas que ofrece un alto rendimiento en lo que al renderizado se refiere, superando a otros motores como ERB y Haml.

3. Liquid 4.0

Liquid es un lenguaje de marcado que se caracteriza por generar plantillas seguras que no puedan afectar al servidor donde van a ser renderizadas. Tiene sus orígenes en el sistema de comercio electrónico Shopify que permite crear tiendas online de manera sencilla y su uso se ha extendido a todo tipo de aplicaciones web. Liquid, al igual que el Slim, es “sin estado”, lo que permite separar la compilación del renderizado.

5.3.3 Motores de Plantillas Javascript

1. Mustache 2.3.0

Mustache está considerado como la base de los motores de plantillas de Javascript y, además, está disponible para la mayoría de otros lenguajes de programación más populares. Su funcionamiento se define como “sin lógica” debido a que únicamente realiza cambios a etiquetas, no contiene bucles ni cláusulas condicionales. Una plantilla Mustache únicamente referencia métodos de la vista. Además ha dado origen a otros motores de Javascript, como Handlebars.

2. EJS: Embedded Javascript 1.0

EJS (Embedded Javascript) es un motor de plantillas que permite trabajar con código Javascript incrustado. Este motor es adecuado para generar plantillas en las que no hay código potencialmente peligroso incrustado. EJS permite ejecutar y/o parsear Javascript tanto en el lado servidor como en el cliente. Además, posibilita el uso de parciales para incluir y renderizar sub plantillas evitando la repetición de código.

3. Jade 1.11

Jade es un motor ligeramente distinto a los anteriores, diseñado para trabajar en el lado de servidor corriendo bajo Node.js. Este motor es distinto a los dos vistos anteriormente a nivel de sintaxis, puesto que hace un uso intensivo de la indentación. A nivel general, es un lenguaje de programación construido sobre HTML que permite crear páginas web de manera más rápida y sencilla, evitando el uso extensivo de etiquetas, unido a la capacidad para el uso de variables y scripts Javascript.

5.3.4 Motores de Plantillas PHP

1. Smarty 3.1.30

Sin duda el motor de plantillas más conocido de PHP es Smarty. Dispone de una gran comunidad de usuarios y ofrece actualizaciones de forma frecuente. El código de Smarty es bastante simple y fácil de comprender. El rendimiento de las aplicaciones que utilizan este motor de plantillas es bastante satisfactorio. Smarty separa el código HTML del código PHP, de forma que en la parte de HTML utiliza variables y tags propias del motor. De esta forma quedan independizadas la parte de front-end y back-end, lo cual es muy deseable de cara al mantenimiento de las aplicaciones y a la resolución de problemas.

2. Twig 2.0

Este motor es bastante reciente, pero también uno de los mejor valorados por los usuarios. En comparación con otros que disponen de más usuarios y soporte por llevar mucho más tiempo, Twig no envidia nada en cuestión de rendimiento, acercándose bastante a los más rápidos y superando a muchos otros. Es un lenguaje rápido porque los templates son compilados a código optimizado de PHP y reducidos al mínimo. Dispone también de un modo sand-box para evaluar la confianza del código lo que le hace más robusto y seguro. Además tiene todo el impulso de la comunidad de Symfony que colabora en el desarrollo.

5.3.5 Motores de Plantillas Python

1. Django 1.11.5

Django, el framework más importante dentro de Python, dispone de su propio motor de plantillas. Está basado en texto, permite definir variables y una lógica básica de control, mediante cláusulas condicionales y bucles y es extensible mediante etiquetas personalizadas a través de código en Python. Django pone énfasis en el re-uso, la conectividad y extensibilidad de componentes, el desarrollo rápido y el principio No te repitas (DRY, del inglés Don't Repeat Yourself).

2. Jinja 2

Jinja es un motor inspirado en el de Django, pero con capacidades extendidas respecto a éste. Permite la ejecución de código Python arbitrario en un entorno controlado mediante sandboxing, un sistema de debug propio y optimizaciones en su compilador a Python para aumentar su rendimiento.

3. Mako 1.0.7

La última librería de plantillas de Python que vamos a ver está basada tanto en Django, como en Jinja 2, como en otros motores de Python. Su API es simple y únicamente involucra el uso de una clase, en tiempo de ejecución se acerca a la velocidad de Jinja2, contiene estructuras de control de flujo basadas en código Python real, permite el filtrado de URL y HTML.

6 Otros elementos en la arquitectura de desarrollo web

6.1 Servidores web

Los servidores web son instrumentos esenciales para el desarrollo de las aplicaciones web. Como se vio al comienzo del tema, estos servicios se sitúan en la máquina servidora y son el software de base sobre el que corren las aplicaciones.

Un servidor web es un programa que implementa el protocolo HTTP (hypertext transfer protocol). Este protocolo está diseñado para transferir lo que llamamos hipertextos, páginas web o páginas HTML, textos complejos con enlaces, figuras, formularios, botones y objetos incrustados como animaciones o reproductores de sonidos.

A continuación se analizarán algunas características de los servidores web más populares por su importancia en el desarrollo de aplicaciones web, para más detalle se recomienda consultar a los temas específicos.

1. Apache

El servidor HTTP Apache es un servidor web de software libre desarrollado por la Apache Software Foundation (ASF). El producto obtenido de este proyecto es un servidor de código fuente completo, descargable y gratuito. Apache es robusto y con un ciclo de desarrollo muy rápido gracias a la gran cantidad de colaboradores voluntarios de que dispone.

Es un servidor web estable ya que su probada robustez impide caídas o cambios en el servidor inesperados. También es flexible y eficiente por ser capaz de trabajar con el estándar HTTP/1.1 (RFC2616) y con la mayor parte de las extensiones web que existen en la actualidad, como son los módulos PHP, SSL, CGI, SSI, proxy,...

Ofrece posibilidad de extensión al disponer de gran cantidad de módulos que amplían su funcionalidad. Y también es multiplataforma ya que está disponible para diferentes plataformas como GNU/Linux, Windows, MacOS.

2. IIS. Internet Information Server

Internet Information Services o IIS es un servidor web y un conjunto de servicios para el sistema operativo Microsoft Windows. Originalmente era parte del Option Pack para Windows NT pero luego fue integrado en otros sistemas operativos de Microsoft. Los servicios que ofrece son: FTP, SMTP, NNTP y HTTP/HTTPS.

Se basa en varios módulos que le dan capacidad para procesar distintos tipos de páginas. Microsoft incluye los de Active Server Pages (ASP) y ASP.NET. También pueden ser incluidos los de otros fabricantes, como PHP o Perl.

3. NGINX

Se trata de un servidor web y proxy inverso de código abierto ligero de alto rendimiento, que también incluye servicios de correo electrónico con acceso al Internet Message Protocol (IMAP) y al servidor Post Office Protocol (POP). NGINX puede ser utilizado como un proxy inverso, y en este modo, NGINX se utiliza para equilibrar la carga entre los servidores back-end, o para proporcionar almacenamiento en caché para un servidor back-end lento. La popularidad de este servidor web ha ido en aumento y empresas punteras de Internet como Facebook o WordPress lo utilizan en sus portales.

Son muchas las características que nos ofrece este servidor web, pero una de las más importantes es que se trata de un software que es asíncrono, a diferencia de Apache que está basada en procesos. La ventaja principal de ser asíncrono, es su escalabilidad. En un sistema basado en procesos, cada conexión simultánea requiere de un hilo, lo que puede llevar a sobrecargar el servidor, mientras que en un servidor asíncrono se gestionan las peticiones en muy pocos hilos, reduciendo las posibilidades de sobrecarga en el servidor.

4. Tomcat

No es propiamente un servidor web sino más bien un servidor de aplicaciones y contenedor de Servlets, desarrollado bajo el proyecto Jakarta en la Apache Software Foundation. Tomcat puede ser usado en combinación con el servidor web Apache para mejorar su desempeño como servidor web. Además, Tomcat implementa las especificaciones de los servlets y de JavaServer Pages (JSP).

6.2 Conexión con bases de datos

La necesidad de almacenar información es cada vez más importante, y se puede decir que los sistemas de bases de datos son unos de los puntos de mayor criticidad en las aplicaciones web. Son estos sistemas los que almacenan y gestionan todos los datos, de carácter público y personal, bajo las condiciones de replicación, seguridad y privacidad que sean necesarios según la naturaleza de los mismos. A continuación se presentan algunas de las bases de datos más comúnmente usadas en las aplicaciones web para pasar a analizar los frameworks más usados para acceder a ellas.

6.2.1 Bases de datos SQL

Las bases de datos SQL son bases de datos relacionales que almacenan información de naturaleza heterogénea y que guardan relaciones que pueden ser representadas formalmente según el modelo relacional. Este modelo pretende representar los objetos de la vida real mediante tablas, relaciones y restricciones entre éstas.

1. Oracle

La solución comercial de Oracle es probablemente la más conocida. Las bases de datos que implementa siguen el modelo de datos relacional, es accesible desde diferentes lenguajes de programación y es soportada por casi todos los sistemas operativos servidores (todas estas características se detallan más adelante en comparación con las del resto de soluciones de base de datos). En cuanto a la arquitectura de las bases de datos Oracle cabe destacar en primer lugar la modularidad o arquitectura en parrilla, que permite utilizar de forma independiente los distintos módulos según las necesidades que se tengan, de forma que el añadir o prescindir de alguno de éstos no interfiere en la configuración actual.

2. Microsoft SQL

La solución de Microsoft sobre bases de datos es comercial como la de Oracle, y es también relacional. Su arquitectura, por tanto, se basa en tablas donde se almacenan los datos, las cuales guardan relaciones entre ellas. Existen componentes similares a la de Oracle, como son las vistas o los ficheros de logs para la consistencia de datos, aunque éstos son tratados de forma diferente, ya que en Oracle son tres ficheros de tamaño fijo que se van rotando y sobrescribiendo si es necesario, en MSSQL es un único fichero que crece de forma indefinida a menos que se ejecuten sentencias de truncado de este fichero y se comience a sobrescribir). Es una solución en continua evolución que cada vez ofrece mejores opciones.

3. DB2

Esta es la solución de IBM que ha evolucionado mucho, en tanto que en sus principios se implementaba en sistemas propietarios y hoy en día existen versiones disponibles para distintas plataformas (Unix, Linux, Windows). Los conceptos básicos en cuanto a las bases de datos son los mismos que los anteriores, aunque existen diferencias de implementación (una instancia puede contener más de una base de datos, en Oracle sólo una, por ejemplo).

4. MySQL

Solución de base de datos relacional con versión gratuita y de pago, que actualmente es propiedad de la compañía de Oracle (anteriormente lo fue de Sun Microsystems). Dispone de las características propias de las bases de datos relacionales y utiliza el modelo de transacciones ACID (Atomicity, Consistency, Isolation and Durability - Atomicidad, Consistencia, Aislamiento y Durabilidad). Lo que se puede destacar de MySQL son los dos motores de base de datos que ofrece, ambos potentes, pero con finalidades diferentes. El primero se llama MyISAM y se trata de un motor que no soporta relaciones de clave foránea (foreign keys) y se perfila como un buen motor para aplicaciones cuyo uso sea mayoritariamente de consultas y no de modificaciones de los datos. En segundo lugar está el

motor de base de datos llamado InnoDB el cual sí que soporta todas las relaciones entre tablas y ofrece muy buenos resultados con aplicaciones que realizan muchas modificaciones de datos y que requieren una gran concurrencia.

5. SQLite

SQLite es un sistema de base de datos relacional open source, y está orientado para el tratamiento de información a pequeña o media escala. Entre las ventajas e inconvenientes de esta solución, además de las propias de los sistemas open source, se encuentran la portabilidad (se trata de un fichero donde reside toda la base de datos), la sencillez de uso, transacciones ACID, rapidez de acceso a datos y consistencia de datos incluso ante fallos del sistema. En cuanto a los inconvenientes, encontramos los problemas de seguridad que un fichero que se puede leer con un editor de textos corriente ofrece.

6.2.2 Frameworks de bases de datos

Dado que es habitual que los frameworks de persistencia o de acceso a base de datos tengan versiones para varias de las tecnologías de servidor que se están viendo en este tema: Java EE, .NET o Ruby, no se hará una distinción por tecnologías sino que se mostrarán los frameworks más utilizados indicando las tecnologías para las que se pueden emplear.

1. MyBatis / iBATIS

iBATIS es un framework de código abierto basado en capas desarrollado por Apache Software Foundation, que se ocupa de la capa de Persistencia, es decir, se sitúa entre la lógica de Negocio y la capa de la Base de Datos. Puede ser implementado en Java o .NET y también existe un port para Ruby on Rails llamado RBatis. iBATIS asocia objetos de modelo (JavaBeans) con sentencias SQL o procedimientos almacenados mediante ficheros descriptores XML, simplificando la utilización de bases de datos. Posteriormente, el equipo de desarrollo decidió continuar el proyecto en Google Code bajo la nueva denominación MyBatis.

Este framework se ocupa de subdividir la capa de Persistencia en tres subcapas. Por un lado la capa de Abstracción será la interfaz con la capa de la lógica de negocio, haciendo las veces de “fachada” entre la aplicación y la persistencia. Por otra parte la capa de Framework de Persistencia será la interfaz con el gestor de base de datos ocupándose de la gestión de los datos mediante un API. Y, finalmente, la capa de Driver se ocupa de la comunicación con la propia Base de Datos utilizando un Driver específico para la misma.

Toda implementación de iBATIS incluye los componentes: Data Mapper, que proporciona una forma sencilla de interacción de datos entre los objetos Java y .NET y bases de datos relacionales, y también el componente, Data Access Object que oculta la persistencia de objetos en la aplicación y proporciona un API de acceso a datos al resto de la aplicación.

2. Hibernate

Hibernate es un framework de mapeo objeto-relacional (ORM) para la plataforma Java inicialmente pero que también está disponible para .Net con el nombre de NHibernate. Este framework facilita el mapeo de atributos entre una base de datos relacional tradicional y el modelo de objetos de una aplicación, mediante archivos declarativos (XML) o anotaciones en los beans de las entidades que permiten establecer estas relaciones.

Hibernate busca solucionar el problema de la diferencia entre los dos modelos de datos coexistentes en una aplicación: el usado en la memoria que es orientación a objetos y el usado en las bases de datos que es modelo relacional. Para lograr esto permite al desarrollador detallar cómo es su modelo de datos, qué relaciones existen y qué forma tienen. Con esta información Hibernate le permite a la aplicación manipular los datos en la base de datos operando sobre objetos, mientras que Hibernate realizará la conversión de los datos de los tipos utilizados por Java y los definidos por SQL. Hibernate genera las sentencias SQL y libera al desarrollador del manejo manual de los datos que resultan de la ejecución de dichas sentencias, manteniendo la portabilidad entre todos los motores de bases de datos con un ligero incremento en el tiempo de ejecución.

Hibernate está diseñado para ser flexible en cuanto al esquema de tablas utilizado, para poder adaptarse a su uso sobre una base de datos ya existente. También tiene la funcionalidad de crear la base de datos a partir de la información disponible. Hibernate ofrece también un lenguaje de consulta

de datos llamado HQL (Hibernate Query Language), al mismo tiempo que una API para construir las consultas programáticamente.

3. JPA: Java Persistence API

Java Persistence API es la API de persistencia desarrollada para la plataforma Java EE. Es un framework del lenguaje de programación Java que maneja datos relacionales en aplicaciones usando la Plataforma Java en sus ediciones Standard (Java SE) y Enterprise (Java EE). El framework se ocupa de tres áreas: el propio API, el lenguaje de consulta Java Persistence Query Language (JPQL) y la definición de metadatos objeto/relacional. El objetivo que persigue el diseño de esta API es no perder las ventajas de la orientación a objetos al interactuar con una base de datos y permitir usar objetos regulares (conocidos como POJOs).

Una entidad de persistencia (entity) es una clase de Java ligera, cuyo estado es persistido de manera asociada a una tabla en una base de datos relacional. Las instancias de estas entidades corresponden a un registro (conjunto de datos representados en una fila) en la tabla. Normalmente las entidades están relacionadas a otras entidades, y estas relaciones son expresadas a través de metadatos objeto/relacional. Los metadatos del objeto/relacional pueden ser especificados directamente en el fichero de la clase, usando las anotaciones de Java (annotations), o en un documento descriptivo XML, el cual es distribuido junto con la aplicación.

4. Entity Framework

Entity Framework es un conjunto de tecnologías dentro del Framework .NET, que permiten el desarrollo de aplicaciones de software orientadas a datos. Permite modelar las entidades, las relaciones y la lógica de los problemas, y también deben trabajar con los motores de datos que se usan para almacenar y recuperar los datos.

Entity Framework permite trabajar con datos en forma de objetos y propiedades específicos del dominio sin tener que preocuparse por las tablas y columnas de la base de datos subyacente donde se almacenan estos datos. Con Entity Framework, se puede trabajar en un nivel mayor de abstracción cuando se trata con datos, y se pueden crear y mantener aplicaciones orientadas a datos con menos código que en las aplicaciones tradicionales. Entity Framework es un componente de .NET Framework, las aplicaciones de Entity Framework se pueden ejecutar en cualquier equipo en el que esté instalado .NET Framework a partir de la versión 3.5.

5. Doctrine

Doctrine es un framework mapeador de objetos-relacional (ORM) escrito en PHP que proporciona una capa de persistencia para objetos PHP. Doctrine requiere muy bajo nivel de configuración para empezar un proyecto, ya que puede generar clases a partir de una base de datos existente y después el programador puede especificar relaciones y añadir funcionalidad extra a las clases autogeneradas. No es necesario generar o mantener complejos esquemas XML de base de datos como en otros frameworks. Doctrine tiene la posibilidad de escribir consultas de base de datos utilizando un dialecto de SQL denominado DQL (Doctrine Query Language) que está inspirado en Hibernate.

Doctrine soporta datos jerárquicos y también dispone de soporte para métodos que pueden validar o modificar las escrituras y lecturas de la base de datos (hooks) y para eventos para manejar la lógica de negocio relacionada. Tiene herencia y cumple con transacciones ACID. Además tiene mejoras de rendimiento como un sistema de cache o una función "compilar" que combina varios archivos PHP del framework en uno solo para evitar el descenso de rendimiento que provoca incluir varios archivos PHP.

6.3 Interconexión con sistemas y servicios

El diseño de las aplicaciones web es cada vez más modular y las aplicaciones se estructuran una serie de componentes o servicios reutilizables. Estos componentes pueden encontrarse distribuidos a lo largo de una serie de máquinas conectadas en red por lo que se requiere que los servicios web mantengan la comunicación entre los componentes. Los servicios web nos permitirán distribuir nuestra aplicación a través de Internet, pudiendo una aplicación utilizar los servicios ofrecidos por cualquier servidor conectado a Internet.

6.3.1 Servicios web

Es cada vez más frecuente que los sistemas no se encuentren aislados si no que se comuniquen e intercambien datos con otros sistemas. Es para este intercambio para el que los servicios web resultan de especial utilidad, al ser componentes a los que se puede acceder usando protocolos web estándar para el intercambio de información en formato XML o JSON.

Los servicios web ofrecen una serie de llamadas públicas que pueden ser invocadas como procedimientos desde Internet o intranet y que son independientes de la implementación ya que la única parte pública es la llamada y los parámetros que funcionan a modo de contrato encapsulando la implementación. Existen dos tipos fundamentales de implementación de los servicios web: servicios web basados en SOAP o servicios web basados en REST.

1. Servicios web SOAP (Simple Object Access Protocol)

Este tipo de servicios web utilizan mensajes XML para comunicarse utilizando el SOAP, de tal manera que el mediante el lenguaje XML se define tanto la arquitectura y formato de los mensajes. La descripción de las llamadas disponibles en el servicio se realiza mediante el protocolo WSDL (Web Services Description Language), que también está basado en XML.

2. Servicios web basado en RESTful (Representational State Transfer Web Service)

Dada la versatilidad y simplicidad de su implementación estos servicios web se han convertido en los más frecuentes en la actualidad. Se suelen implementar sobre HTTP y no requieren mensajes XML o definiciones del servicio en forma de fichero WSDL ya que los interfaces siguen definiciones normalizadas basadas en estándares muy conocidos como HTTP, SML, URI o MIME.

6.3.2 Frameworks de web services

Se van a analizar por último algunos frameworks específicos de acceso a servicios web, aunque hay que tener en cuenta que algunos de los frameworks que se han analizado en los epígrafes anteriores ya incluyen herramientas para acceder de forma sencilla a servicios web, como por ejemplo: CodeIgniter, .NET Framework o Zend Framework.

1. Apache Axis / Axis2

Apache Axis es un framework de código abierto, basado en XML para servicios web que es desarrollado en la Apache Software Foundation. Consiste en una implementación en Java y otra en C++ del servidor SOAP. Incluye también diversas herramientas y APIs para generar y desplegar aplicaciones de servicios web. Mediante el uso Axis se pueden crear aplicaciones interoperables y distribuidas.

Apache Axis2 es un framework y un motor para servicios web. En esta segunda versión se hizo un rediseño total y una reimplementación completa. También existen implementaciones de Axis2 en Java y en C. Axis2 no solo provee la capacidad de agregar servicios web a las aplicaciones web, sino que además puede funcionar como servidor autónomo. Apache Axis2 soporta SOAP 1.1 y SOAP 1.2, y también integra soporte para el estilo REST para servicios web.

2. Apache CXF

Apache CXF es un framework completo, de código abierto para servicios web. Se originó como combinación de dos proyectos de código abierto: Celtix y XFire. Estos proyectos fueron combinados por personas que trabajaban juntas en Apache Software Foundation. El nombre CXF se deriva de la combinación de los nombres de proyecto "Celtix" y "XFire".

CXF cuenta con una separación limpia entre los front-ends y el código fuente central. Además ofrece un alto rendimiento y gran simplicidad en la creación de clientes y endpoints sin necesidad de anotaciones. CXF frecuentemente se emplea en conjunto con Apache ServiceMix, Apache Camel y Apache ActiveMQ en proyectos de infraestructura con arquitecturas orientadas a servicios (SOA).

3. JAX-RPC / JAX-WS

JAX-WS (Java API for XML Web Services) es una interfaz de programación de aplicaciones de Java en XML para la creación de servicios web. Es parte de la plataforma Java EE de Sun Microsystems. Al igual que las otras API de Java EE, JAX-WS utiliza anotaciones para simplificar el desarrollo y

despliegue de los clientes y puntos finales de servicios web. JAX-WS 2.0 reemplazó a la API JAX-RPC en Java EE 5.

La implementación de referencia de JAX-WS se desarrolla como un proyecto de código abierto y forma parte del proyecto GlassFish, un servidor de aplicaciones Java EE de código abierto. Se llama JAX-WS RI (por Reference Implementation) y se dice que es la implementación de calidad de producción (contrariamente a la implementación de referencia antigua que era una prueba de concepto).

4. JAX-RS

Java API for RESTful Web Services (JAX-RS) es una API del lenguaje de programación Java que proporciona soporte en la creación de servicios web de acuerdo con el estilo arquitectónico Representational REST. JAX-RS usa anotaciones para simplificar el desarrollo y despliegue de los clientes y puntos finales de los servicios web. A partir de la versión 1.1 en adelante, JAX-RS es una parte oficial de Java EE. Una característica notable de ser parte oficial de Java EE es que no se requiere configuración para comenzar a usar JAX-RS. Para los entornos inferiores a Java EE 6 se requiere una (pequeña) entrada en el descriptor de despliegue web.xml.

RESUMEN

Las aplicaciones web son aquellas aplicaciones que los usuarios pueden utilizar accediendo a un servidor web a través de Internet o de una intranet **mediante un navegador**.

Elementos de la arquitectura cliente-servidor

- **Cliente**
 - Inicia solicitudes o peticiones. Tiene un papel activo en la comunicación.
 - Espera y recibe las respuestas del servidor.
 - Puede conectarse a varios servidores a la vez.
 - Interactúa directamente con los usuarios finales mediante una interfaz gráfica de usuario.
- **Servidor**
 - Al iniciarse espera a que le lleguen las solicitudes de los clientes.
 - Tras la recepción de una solicitud, la procesan y luego envían la respuesta al cliente.
 - Aceptan conexiones desde un gran número de clientes.
 - No es frecuente que interactúen directamente con los usuarios finales.

Ventajas de la arquitectura web cliente-servidor

- No requiere instalar software especial (en los clientes).
- Bajo coste en actualizar los equipos con una nueva versión.
- Acceso a la última y mejor versión.
- Información centralizada.
- Seguridad y copias de seguridad.
- Movilidad.
- Reducción de costes en los puestos cliente (mayor longevidad).

Arquitectura en capas

- Arquitectura en 2 capas
- Arquitectura en 3 capas
 - Capa de **presentación**
 - Es responsable de la presentación de los datos, recibiendo los eventos de los usuarios y controlando la interfaz de usuario.
 - Capa de **lógica** de negocios
 - Esta capa no está presente en la arquitectura en 2 capas en forma explícita.
 - Los objetos de negocios que implementan las reglas de negocios “viven” aquí, y están disponibles para la capa de presentación
 - Protege del acceso directo a la información desde la capa de presentación
 - Capa de **persistencia**
 - Es responsable del almacenamiento de los datos
 - Es común reusar sistemas existentes de bases de datos en esta capa
 - Actualmente se usan manejadores relacionales: son avanzados, permiten el uso de triggers y paquetes. Existen manejadores Orientados a Objetos

Arquitectura P2P

Todos los nodos se comportan como iguales entre sí actuando simultáneamente como clientes y servidores respecto a los demás nodos de la red. Permiten el intercambio directo de información entre los nodos interconectados.

Características:

- Escalabilidad
- Robustez
- Descentralización
- Distribución de costes
- Seguridad

Front-end

Las tecnologías que se sitúan en este extremo de la arquitectura están orientadas a formatear la información que se ha recibido de una petición en concreto al servidor, pero también se encargan de mantener la información entre las distintas peticiones constituyendo las sesiones.

Tecnologías: HTML, XML, CSS, Adobe Flash, Javascript, ECMAScript, AJAX, JScript, VBScript, Applets, Controles ActiveX

Framework: estructura de soporte definida sobre la que otras aplicaciones software pueden ser organizadas y desarrolladas. Puede incluir programas soporte, librerías y scripts de código y cualquier tipo de servicio que permita unir los diferentes componentes de una aplicación.

Frameworks de Javascript

- jQuery 3.2.1
- jQuery Mobile 1.4.5
- Bootstrap 4
- AngularJS 1.6.6
- EmbedJS 4.2.1
- BackboneJS 1.3.3

Experiencia de Usuario. UX.

Experiencia de usuario

La experiencia de usuario es el conjunto de factores y elementos relativos a la interacción del usuario, con un entorno o dispositivo concretos, cuyo resultado es la generación de una percepción positiva o negativa de dicho servicio, producto o dispositivo.

Elementos de las UX:

- ✓ Especificaciones funcionales
- ✓ Diseño de interacción
- ✓ Arquitectura de la información
- ✓ Diseño de información
- ✓ Diseño de navegación

Usabilidad

La usabilidad se refiere a la claridad y la elegancia con que se diseña la interacción con un programa de ordenador o un sitio web.

Usabilidad web

La usabilidad web es la disciplina que estudia la forma de diseñar sitios web para que los usuarios puedan interactuar con ellos de la forma más fácil, cómoda e intuitiva posible.

Reglas de usabilidad web

- ✓ Rápido
- ✓ Simple
- ✓ Investigable
- ✓ Para la mayoría
- ✓ Manténgalo actualizado

Arquitecturas aplicación web

- Página web **estática**
 - El navegador hace petición al servidor mediante HTTP
 - El servidor transforma URL a ruta en disco
- Página web **interactiva**
 - El contenido de la página web está alojado en el disco duro del servidor (estático)
 - El cliente es dinámico porque las páginas incluyen código Javascript que se ejecuta en el navegador
 - Aplicación web **interactiva**: El Javascript se utiliza para crear efectos gráficos
 - Aplicación web con **AJAX**: Javascript se usa para no tener que recargar completamente la página al pulsar un link. La página se puede cargar por partes, primero la información importante y en segundo plano otros elementos complementarios.
 - Aplicación web **SPA (Single Page Application)**: Existe una única página cuyo contenido va cambiando según el usuario interactúa con botones, pestañas, etc.

Desarrollo web en servidor

El extremo del servidor es la parte más compleja y pesada de la arquitectura. Debe ser capaz de atender las demandas variables de los usuarios, en tiempos que sean adecuados para poder trabajar.

- **Tecnologías:** PHP 7.1.9, Java EE 8, ASP 3.0, ASP.NET 5.0, Ruby 2.4.1 y Python3.6.2
- **Frameworks** Back-end
 - **FRAMEWORKS JAVA EE**
 - ✓ SPRING 4.3.10
 - ✓ APACHE TRAPESTRY 5
 - ✓ SPARK FRAMEWORK 2.6
 - **FRAMEWORKS DE RUBY**
 - ✓ RUBY ON RAILS 5.1.3
 - ✓ SINATRA 2.0
 - ✓ SEQUEL 5
 - **FRAMEWORKS DE JAVASCRIPT**
 - ✓ NODE.JS 8.4
 - ✓ EXPRESS 4.15.4
 - ✓ METEOR 1.5
 - **FRAMEWORKS DE PHP**
 - ✓ SYMFONY 3.3.2
 - ✓ ZEND FRAMEWORK 3.0
 - ✓ LARAVEL 5.5.0
 - ✓ CODEIGNITER 3.1.5
 - **FRAMEWORKS DE PYTHON**
 - ✓ DJANGO 1.11.4
 - ✓ WEB2PY 2.14.6
- **Motores de Plantillas**
 - **MOTORES DE PLANTILLAS JAVA EE**
 - ✓ JSP: JAVA SERVER PAGES 2.3
 - ✓ SPRING TEMPLATE 4.3.10
 - ✓ JSF: JAVA SERVER FACES 2.3
 - **MOTORES DE PLANTILLAS RUBY**
 - ✓ HAML 5.0.3
 - ✓ SLIM 3.0.8
 - ✓ LIQUID 4.0
 - **MOTORES DE PLANTILLAS JAVASCRIPT**
 - ✓ MUSTACHE 2.3.0
 - ✓ EJS: EMBEDDED JAVASCRIPT 1.0
 - ✓ JADE 1.11
 - **MOTORES DE PLANTILLAS PHP**
 - ✓ SMARTY 3.1.30
 - ✓ TWIG 2.0
 - **MOTORES DE PLANTILLAS PYTHON**
 - ✓ DJANGO 1.11.5
 - ✓ JINJA 2
 - ✓ MAKO 1.0.7
- **Bases de Datos SQL:** Oracle, Microsoft SQL, DB2, MySQL, SQLite
 - **FRAMEWORKS BASES DE DATOS**
 - ✓ MyBatis / iBATIS
 - ✓ HIBERNATE
 - ✓ JAP: JAVA PERSISTENCE API
 - ✓ ENTITY FRAMEWORK
 - ✓ DOCTRINE
- **Conexión a servicios web**
 - **FRAMEWORK CONEXION SERVICIOS WEB**
 - ✓ AXIS / AXIS 2
 - ✓ APACHE CFX
 - ✓ JAX-RPC / JAX-WS
 - ✓ JAX-RS

PREGUNTAS DE TEST

- 1) Cuál de los siguientes no es un lenguaje de programación utilizado en el Back-end
 - a) Java
 - b) Ruby
 - c) Javascript
 - d) Todos se puede usar.
- 2) Que tecnología se usa para comunicar de forma asíncrona lanzando peticiones desde el navegador mediante Javascript:
 - a) AJAX
 - b) PHP
 - c) ASP.NET
 - d) JSP
- 3) Que librería Javascript permite la creación sencilla de interfaces adaptativos:
 - a) Angular.js
 - b) jQuery Mobile
 - c) Bootstrap.js
 - d) DataTables
- 4) Cuál de las siguiente SI es un motor de plantillas Python:
 - a) SLIM
 - b) MUSTACHE
 - c) JADE
 - d) DJANGO
- 5) Que tecnología de las siguientes se usa en cliente y en servidor:
 - a) Ruby
 - b) Java
 - c) Javascript
 - d) PHP
- 6) Cuál es la mejor definición de node.js
 - a) Una librería Javascript
 - b) Un paradigma de programación nuevo
 - c) Una maquina virtual de Javascript
 - d) Una plataforma construida sobre el motor de Javascript de Chrome para construir fácilmente aplicaciones rápidas y escalables.
- 7) Cuál de los siguientes elementos no se usa para comunicación con servicios de otras maquinas:
 - a) Servicios web SOAP
 - b) Servicios web RestFul
 - c) Websockets
 - d) DAO

SOLUCIONES A LAS PREGUNTAS DE TEST

PREGUNTA	SOLUCIÓN
1	d
2	a
3	c
4	d
5	c
6	d
7	d

BIBLIOGRAFÍA

1. Conceptos generales de la arquitectura de aplicaciones web. Editorial RA-MA - Autor: Juan Ferrer Martínez
2. Desarrollo de aplicaciones web en el entorno servidor. ic editorial – Autor: Rafael Luis Granados La Paz
3. REALIZACIÓN DE UN SISTEMA WEB COMPLEJO PARA USO DIDÁCTICO. TRABAJO FIN DE MÁSTER. Autor: Daniel Benítez Águila
4. DISEÑO DE APLICACIONES WEB - Facultad de Informática UPM - Antonio Latorre
5. HTTP: The Definitive Guide. O'Reilly & Associates, Inc. / Autores: David Gourley (Author), Brian Totty, Marjorie Sayer, Anshu Aggarwal, Sailu Reddy
6. Desarrollo de una aplicación web para compartir medio de transporte con AngularJS - TRABAJO FIN DE GRADO - Autor: Fco Javier Avilés López
7. Wikipedia
Aplicación web: https://es.wikipedia.org/wiki/Aplicaci%C3%B3n_web
Cliente-servidor: <https://es.wikipedia.org/wiki/Cliente-servidor>
Peer-to-Peer: <https://es.wikipedia.org/wiki/Peer-to-peer>
8. Introducción a los Servicios Web – Departamento de Ciencia de la Computación e IA - Universidad de Alicante: <http://www.jtech.ua.es/j2ee/publico/servc-web-2012-13/sesion01-apuntes.html>